



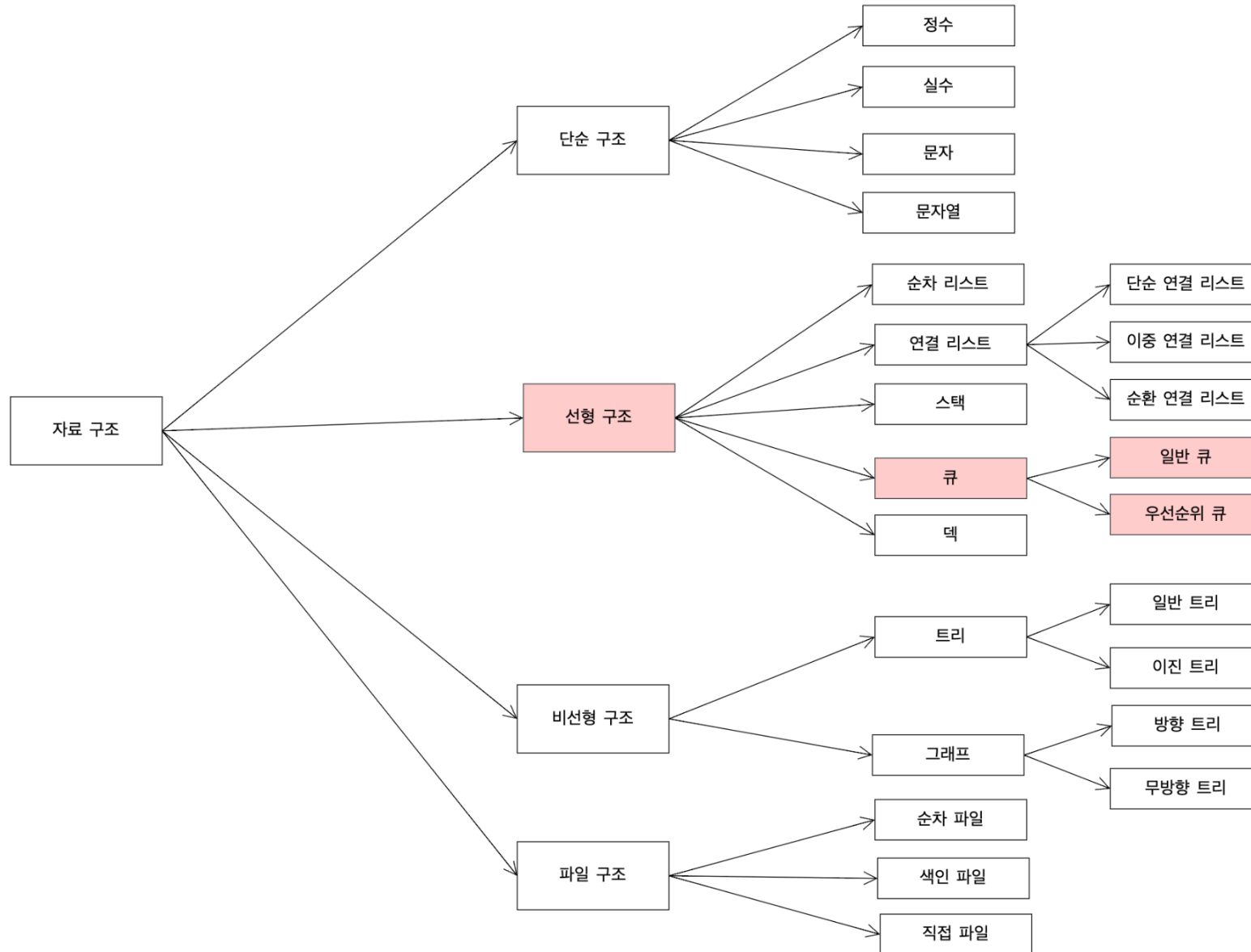
국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

데이터처리특론

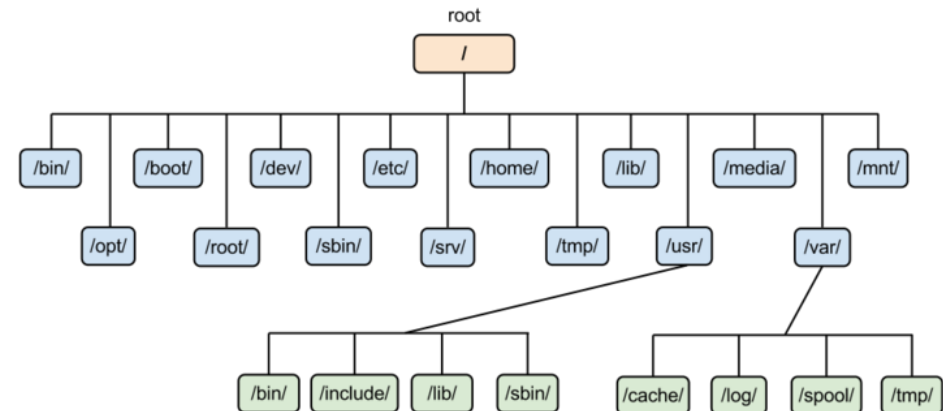
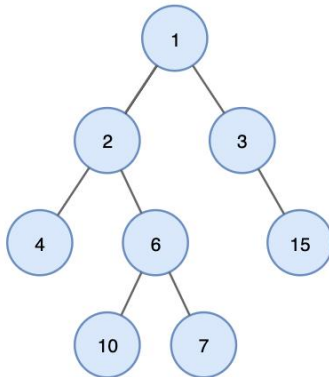


Data structure



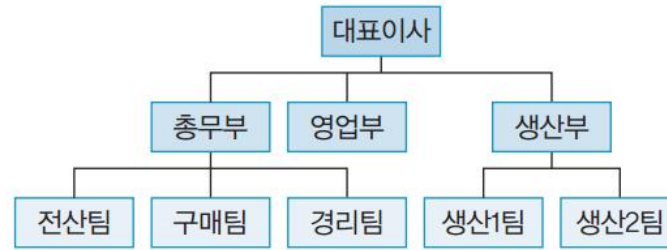
Tree

- 트리 (Tree)란 노드들이 나무 가지처럼 연결된 비선형 계층적 자료구조다.
- 트리는 다음과 같이 나무를 거꾸로 뒤집어 놓은 모양과 유사하다.
- 트리는 또한 트리 내에 다른 하위 트리가 있고 그 하위 트리 안에는 또 다른 하위 트리가 있는 재귀적 자료구조이기도 하다.
- 컴퓨터의 direcory구조가 트리 구조의 대표적인 예가 될 수 있다.
- 트리 구조(tree 構造, 문화어: 나무구조)란 그래프의 일종으로, 한 노드에서 시작해서 다른 정점들을 순회하여 자기 자신에게 돌아오는 순환이 없는 연결 그래프이다.

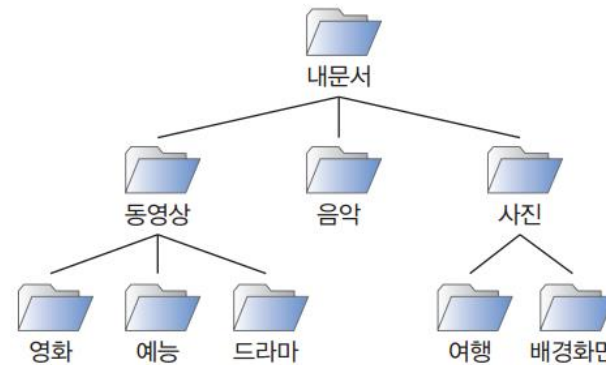


Tree

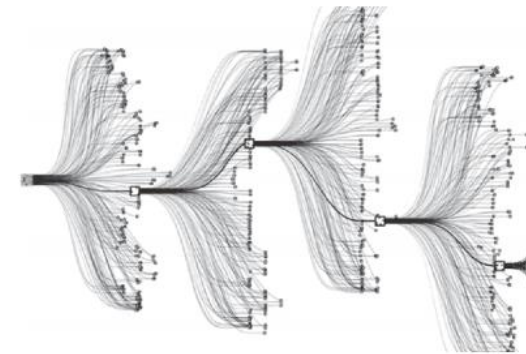
- 일반적인 트리(General Tree)는 실제 트리를 거꾸로 세워 놓은 형태의 자료구조
- 계층적인 자료의 표현에 적합한 자료 구조



(a) 회사의 조직도

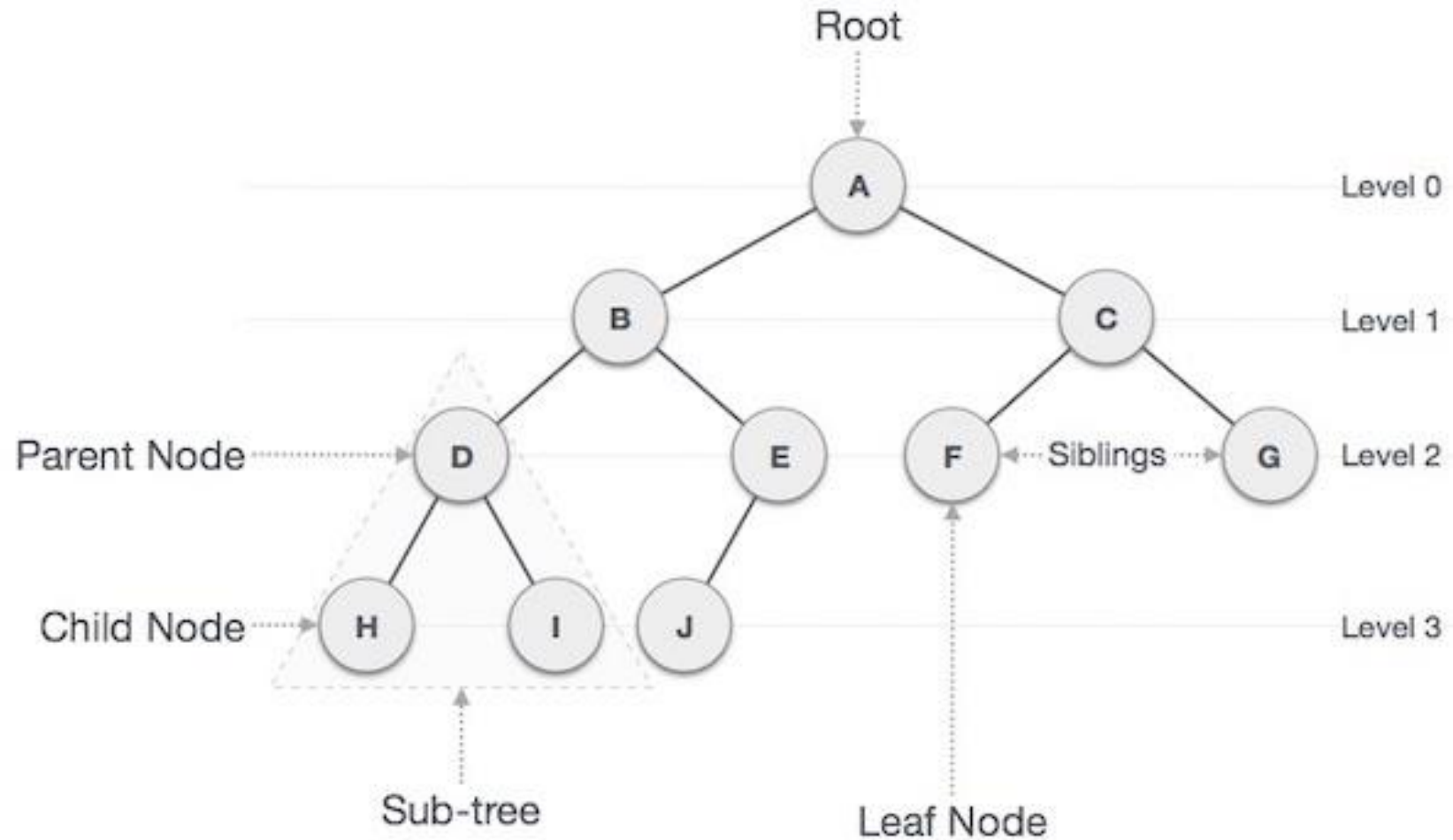


(b) 컴퓨터의 폴더 구조



(c) 인공지능 바둑 프로그램의
거대한 결정 트리(decision tree)

Tree 용어



Tree 용어

- 노드 (Node)
 - ✓ 트리를 구성하고 있는 기본 요소
 - ✓ 노드에는 키 또는 값과 하위 노드에 대한 포인터를 가지고 있음.
 - ✓ A, B, C, D, E, F, G, H, I, J
- 간선 (Edge)
 - ✓ 노드와 노드 간의 연결선
- 루트 노드 (Root Node)
 - ✓ 트리 구조에서 부모가 없는 최상위 노드
 - ✓ root node : A
- 부모 노드 (Parent Node)
 - ✓ 자식 노드를 가진 노드
 - ✓ H, I에 부모 노드는 D
- 자식 노드 (Child node)
 - ✓ 부모 노드의 하위 노드
 - ✓ 노드 D의 자식 노드는 H, I
- 형제 노드 (Sibling node)
 - ✓ 같은 부모를 가지는 노드
 - ✓ H, I는 같은 부모를 가지는 형제 노드

Tree 용어

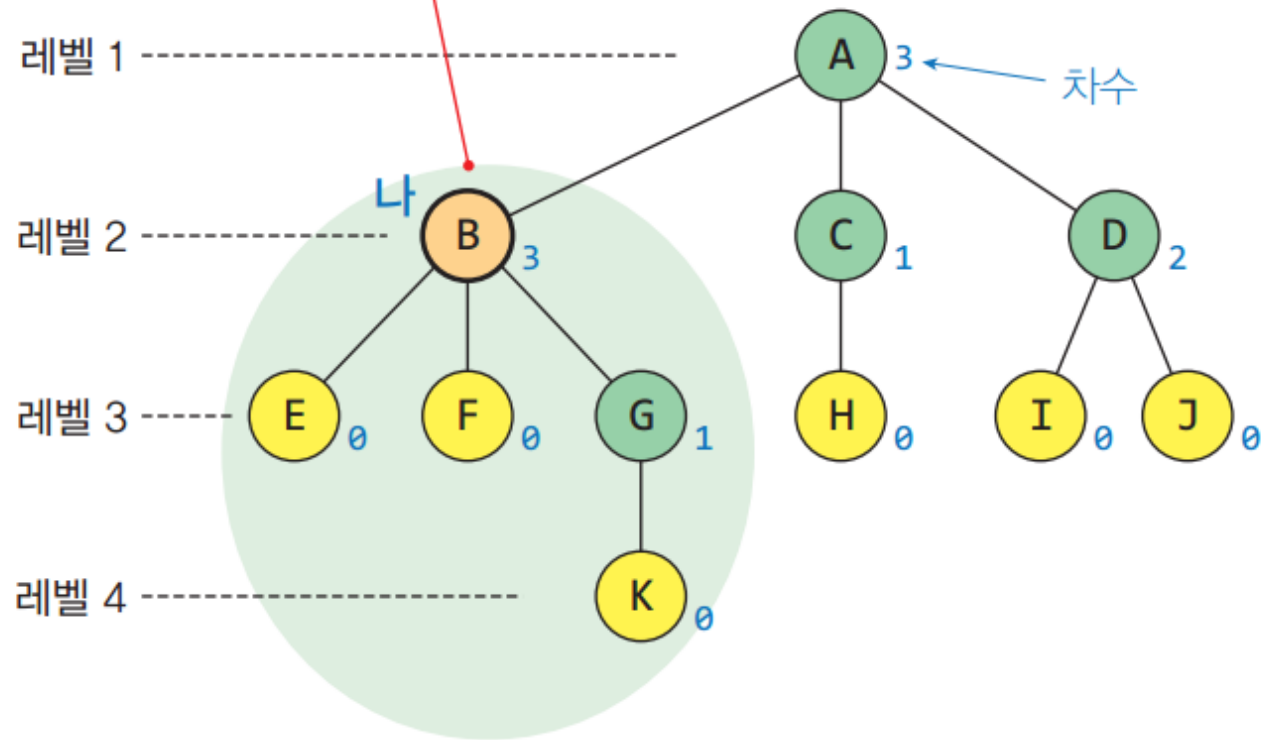
- 외부 노드(external node, outer node), 단말 노드 (terminal node), 리프 노드(leaf node)
 - ✓ 자식 노드가 없는 노드
 - ✓ H, I, J, F, G
- 내부 노드 (internal node, inner node), 비 단말 노드 (non-terminal node), 가지 노드 (branch node)
 - ✓ 자식 노드 하나 이상 가진 노드
 - ✓ A, B, C, D, E
- 깊이 (depth)
 - ✓ 루트에서 어떤 노드까지의 간선(Edge) 수
 - ✓ 루트 노드의 깊이 : 0
 - ✓ D의 깊이 : 2
- 높이 (height)
 - ✓ 어떤 노드에서 리프 노드까지 가장 긴 경로의 간선(Edge) 수
 - ✓ 리프 노드의 높이 : 0
 - ✓ A 노드의 높이 : 3
- Order
 - ✓ 부모 노드가 가질 수 있는 최대 자식의 수
 - ✓ Order 3 : 부모 노드는 최대 3명의 자식을 가질 수 있다.

Tree 용어

- Level
 - ✓ 루트에서 어떤 노드까지의 간선(Edge) 수
- Degree
 - ✓ 노드의 자식 수
 - ✓ Leaf node의 degree : 0; A의 degree : 2
- Path
 - ✓ 한 노드에서 다른 한 노드에 이르는 길 사이에 놓여있는 노드들의 순서
 - ✓ A & H 경로 : A-B-D-H
- Path Length
 - ✓ 해당 경로에 있는 총노드의 수
 - ✓ A & H 경로 길이 : 4
- Size
 - ✓ 자신을 포함한 자손의 노드 수
 - ✓ 노드 B의 size : 6
- Width
 - ✓ 레벨에 있는 노드 수
 - ✓ Level 2 width : 4
- Breadth
 - ✓ 리프 노드의 수
 - ✓ Breadth : 5
- Distance
 - ✓ 두 노드 사이의 최단 경로에 있는 간선(Edge)의 수
 - ✓ D와 J의 Distance : 3

Tree 용어

트리의 모든 노드는 자신의 서브트리의 루트 노드입니다.



- 루트 노드: A
- B의 부모노드: A
- B의 자식 노드: E, F, G
- B의 자손 노드: E, F, G, K
- K의 조상 노드: G, B, A
- B의 형제 노드: C, D
- B의 차수: 3
- 단말 노드: E, F, K, H, I, J
- 비단말 노드: A, B, C, D, G
- 트리의 높이: 4
- 트리의 차수: 3

Tree의 표현 방법

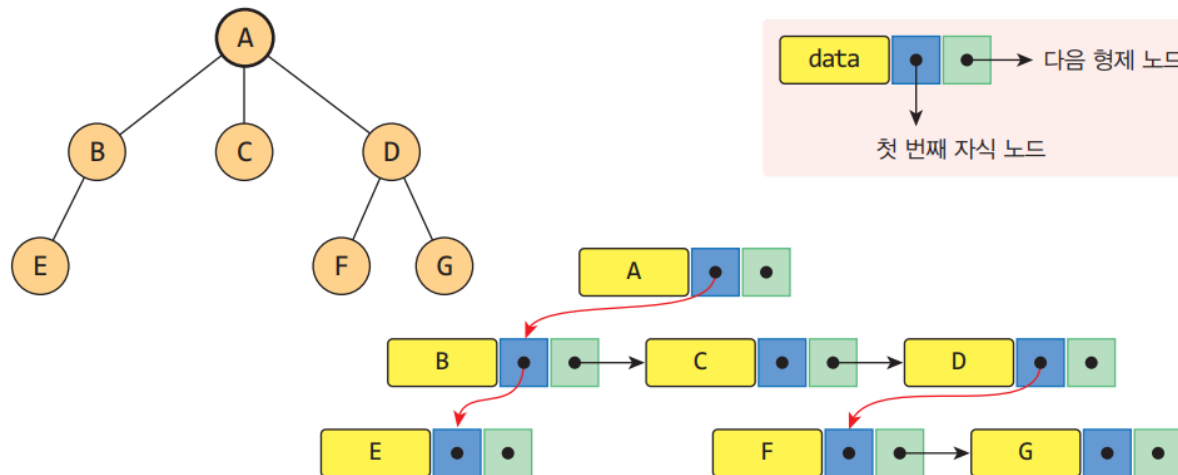
• 일반 트리

- ✓ 일반적인 트리를 메모리에 저장하려면 각 노드에 키와 자식 수만큼의 레퍼런스 저장 필요
- ✓ 노드의 최대 차수가 N이라면, N개의 레퍼런스 필드를 다음과 같이 선언해야 함



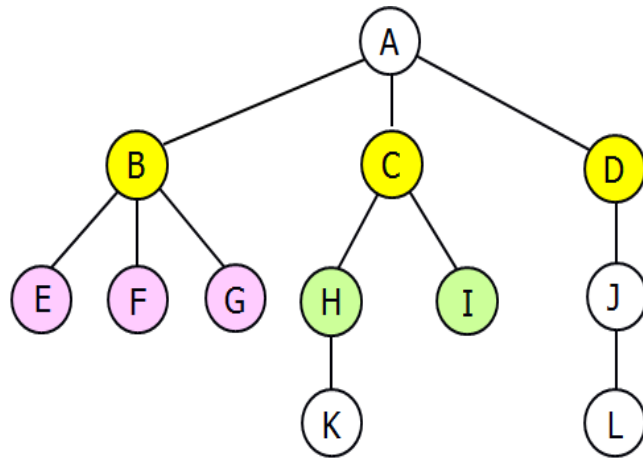
• 왼쪽 자식-오른쪽 형제(Left Child-Right Sibling) 표현

- ✓ 노드의 왼쪽 자식과 왼쪽 자식의 오른쪽 형제를 가리키는 2개의 레퍼런스만을 사용

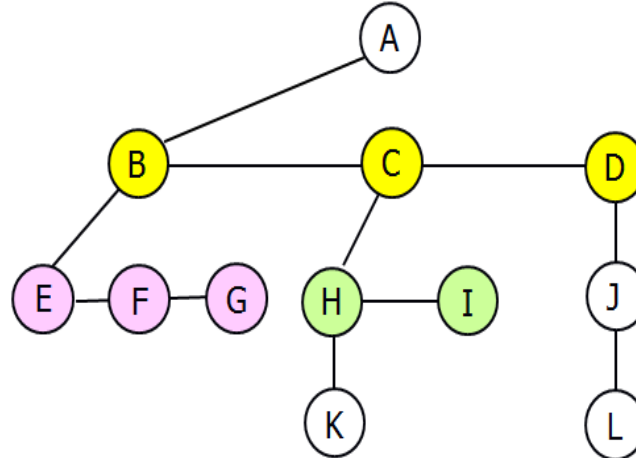


Tree의 표현 방법

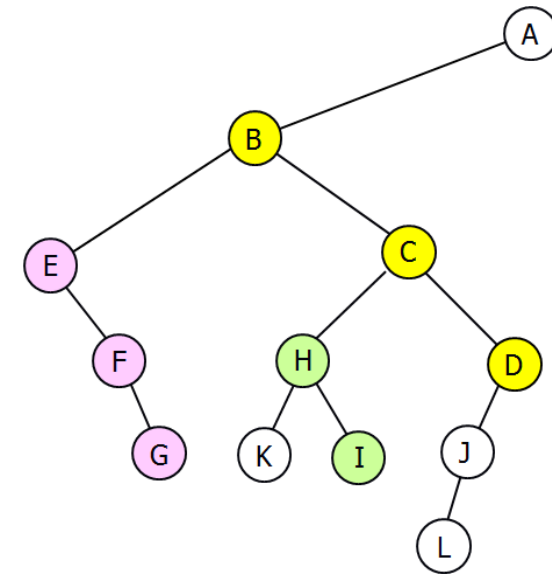
- 왼쪽 자식-오른쪽 형제(Left Child-Right Sibling) 표현
 - ✓ (a)의 트리를 왼쪽 자식-오른쪽 형제 표현으로 변환하면, (b)의 트리를 얻으며, (c)는 (b)의 트리를 45° 시계 방향으로 회전시킨 것



(a)



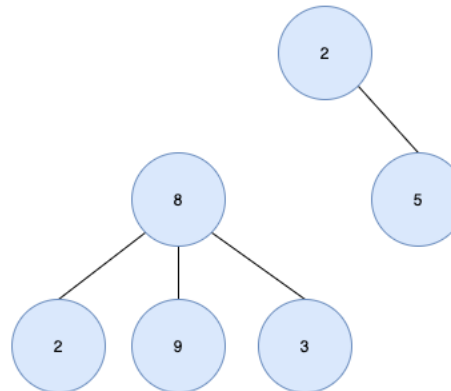
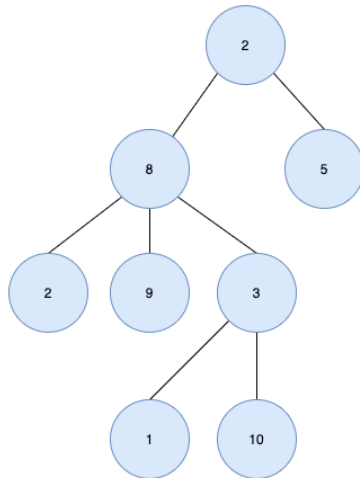
(b)



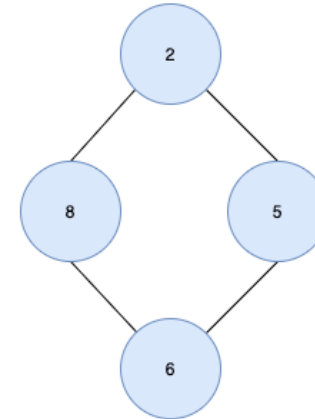
(c)

Tree의 특징

- 하나의 루트 노드와 0개 이상의 하위 트리로 구성되어 있다.
- 데이터를 순차적으로 저장하지 않기 때문에 비선형 자료구조이다.
- 트리내에 또 다른 트리가 있는 재귀적 자료구조이다.
- 단순 순환(Loop)을 갖지 않고, 연결된 무방향 그래프 구조다.
- 노드 간에 부모 자식 관계를 갖고 있는 계층형 자료구조이며 모든 자식 노드는 하나의 부모 노드만 갖는다.
- 노드가 n 개인 트리는 항상 $n-1$ 개의 간선(edge)을 가진다.



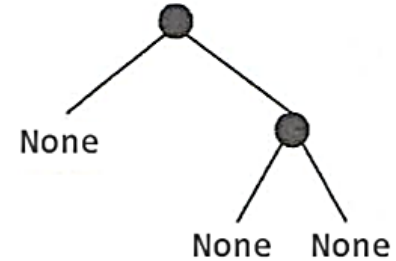
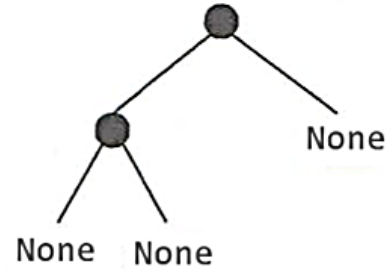
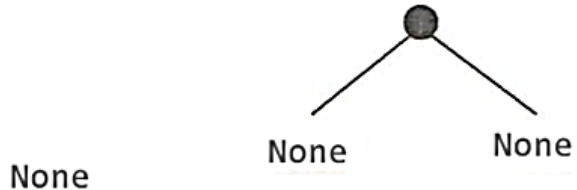
루트 노드가 2개(2, 8)



노드 6에는 2명의 부모 노드(8,5)
가 있고 사이클(2-8-6-5)이 형성

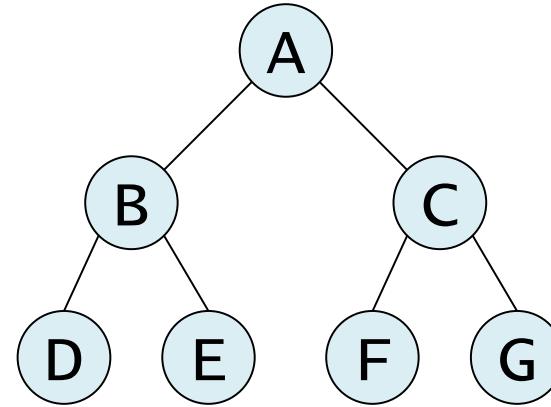
이진트리(binary tree)

- 이진트리(Binary Tree): 각 노드의 자식 수가 2 이하인 트리
- 컴퓨터 분야에서 널리 활용되는 기본적인 자료구조
- 이진트리가 데이터의 구조적인 관계를 잘 반영하고, 효율적인 삽입과 탐색을 가능하게 하며, 이진트리의 서브트리를 다른 이진트리의 서브트리와 교환하는 것이 쉽기 때문
- 이진트리에 대한 용어는 일반적인 트리에 대한 용어와 동일
- **[정의]** 이진트리는 empty이거나, empty가 아니면, 루트노드와 2개의 이진트리인 왼쪽 서브트리와 오른쪽 서브트리로 구성된다.

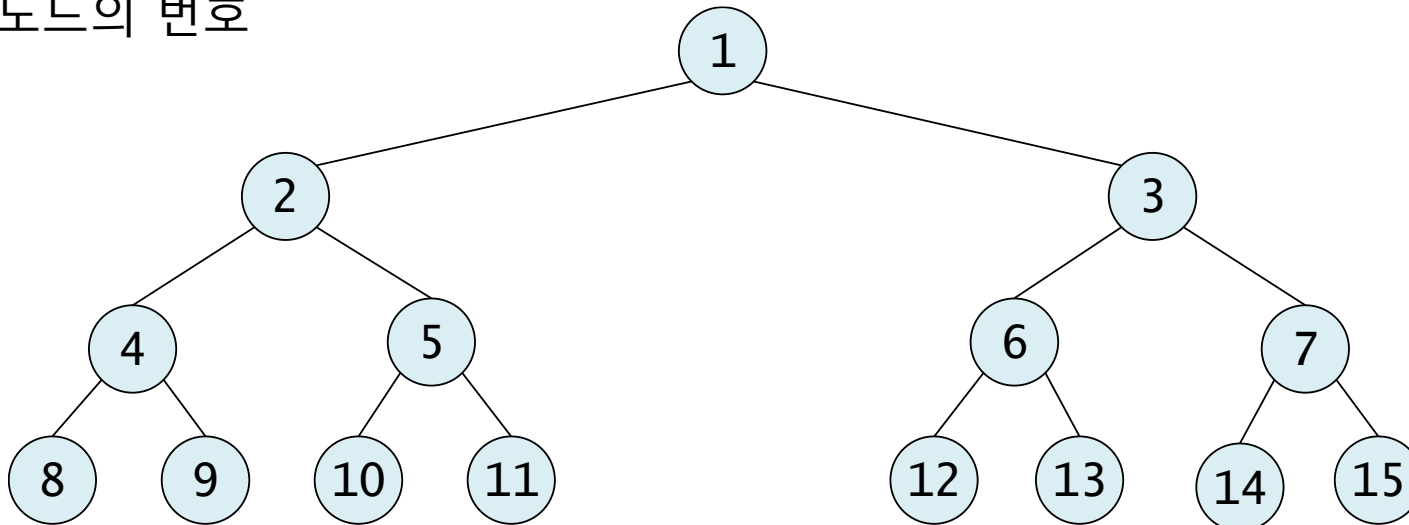


이진트리(binary tree)의 분류

- 포화 이진 트리(full binary tree)
 - 트리의 각 레벨에 노드가 꽉 차있는 이진트리

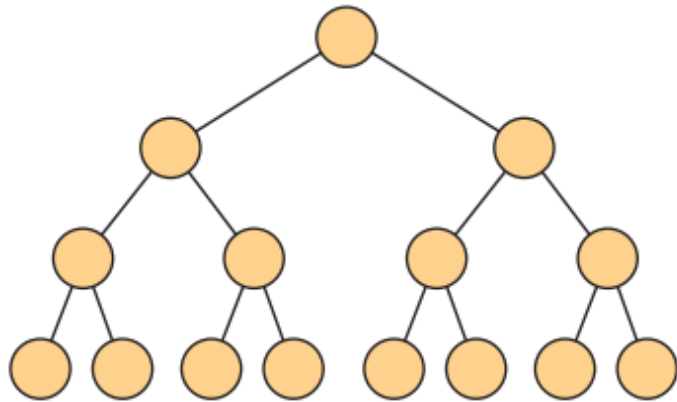
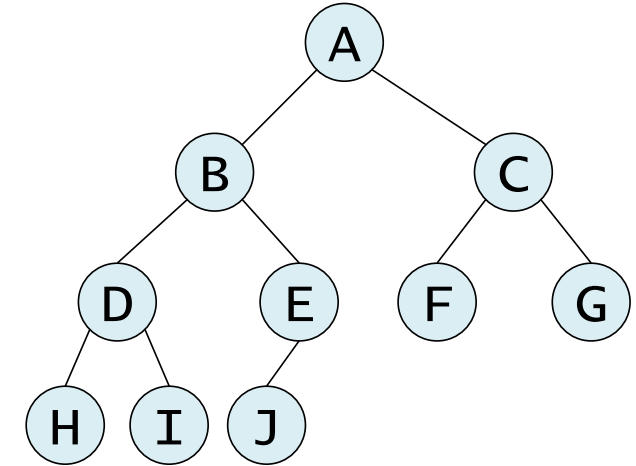


- 노드의 번호

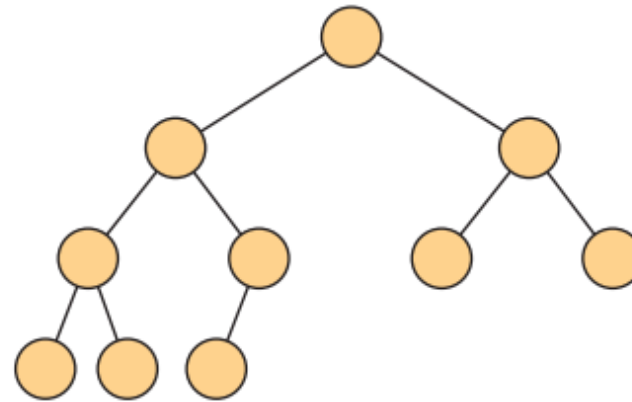


이진트리(binary tree)의 분류

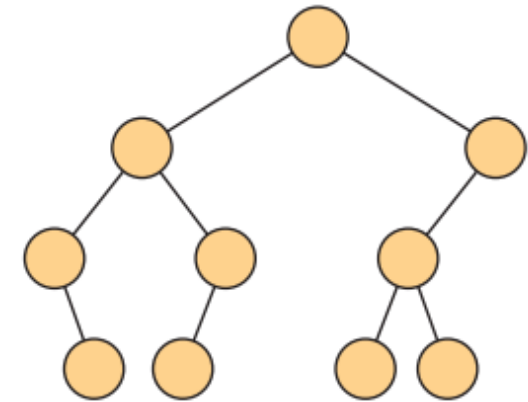
- 완전 이진 트리(complete binary tree)
 - 높이가 h 일 때 레벨 1부터 $h-1$ 까지는 노드가 모두 채워짐
 - 마지막 레벨 h 에서는 노드가 순서대로 채워짐



포화이진트리



완전 이진트리

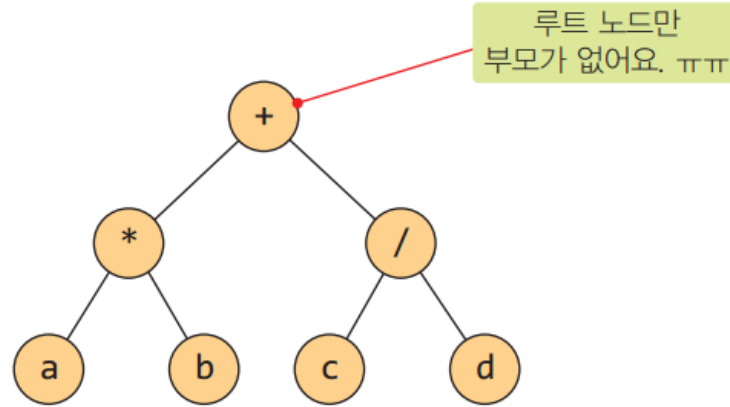


기타이진트리

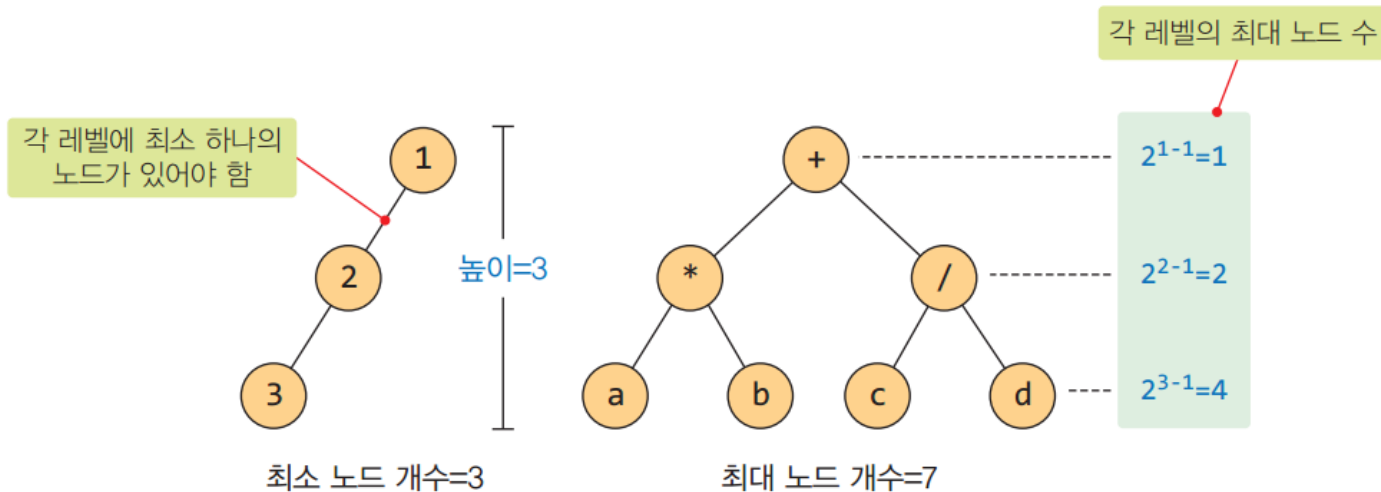
이진트리(binary tree)의 성질

- 노드의 개수가 n 개이면 간선의 개수는 $n-1$

노드의 개수: 7
간선의 개수: 6

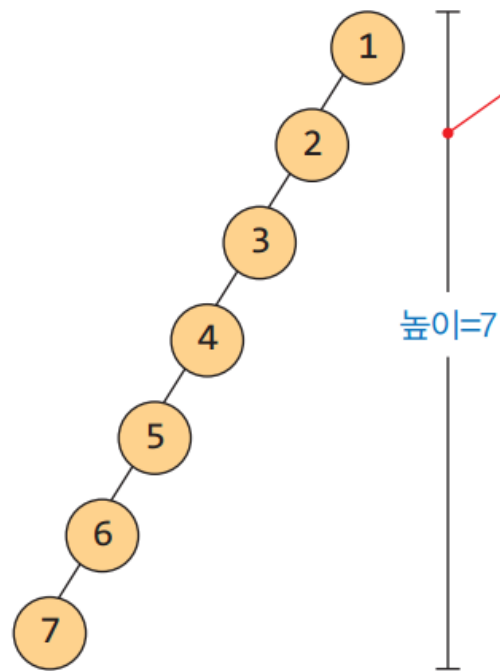


- 높이가 h 이면 $h \sim 2^h - 1$ 개의 노드를 가짐

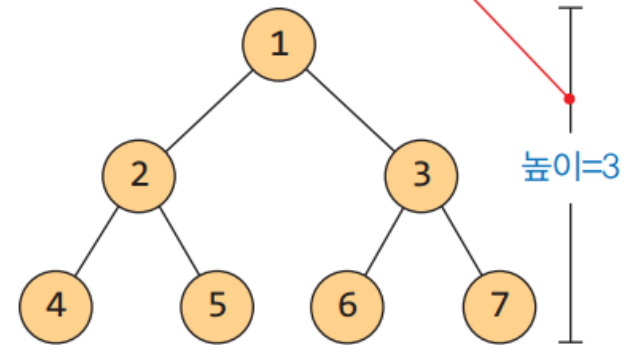


이진트리(binary tree)의 성질

- n 개 노드의 이진 트리 높이: $\lceil \log_2(n + 1) \rceil \sim n$



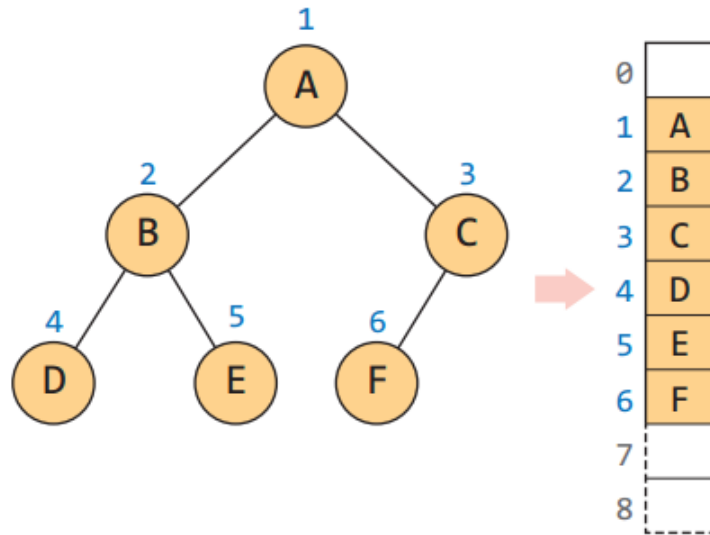
7개의 노드를 일렬로 늘어놓으면 트리의 높이가 최대인 7이 됩니다.



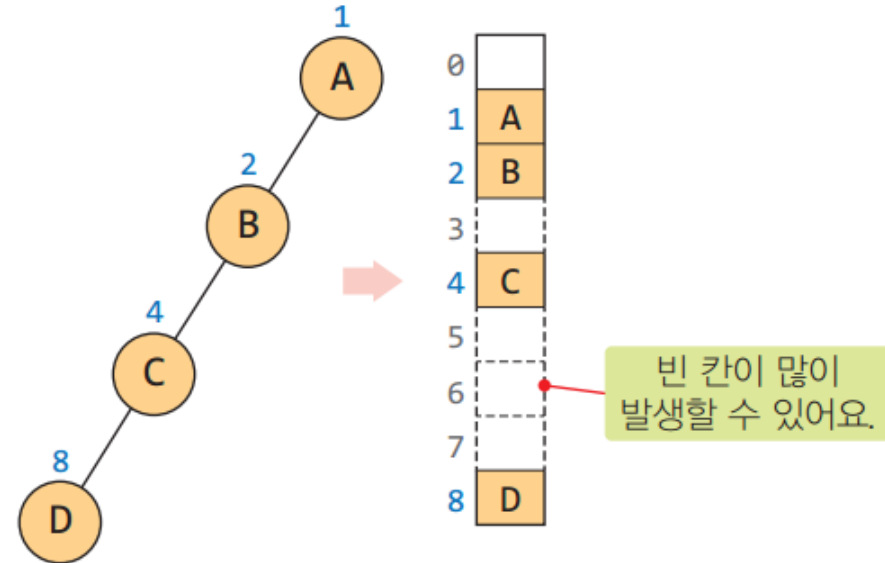
7개의 노드를 루트부터 빠르게 채우면 높이 3인 트리를 만들 수 있어요.

이진트리(binary tree)의 표현

- 배열 표현법



완전이진트리의 배열 표현



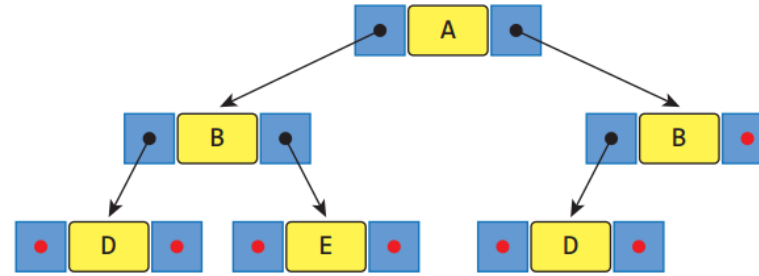
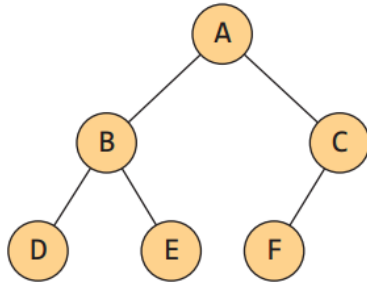
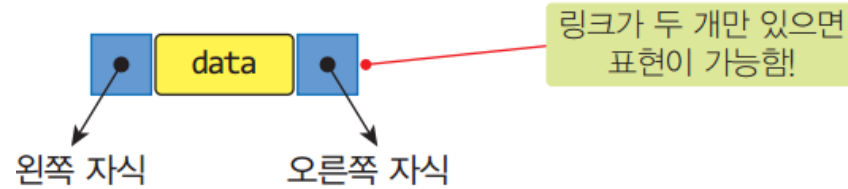
경사이진트리의 배열 표현

- 노드 i 의 부모 노드 인덱스 = $i/2$
- 노드 i 의 왼쪽 자식 노드 인덱스 = $2i$
- 노드 i 의 오른쪽 자식 노드 인덱스 = $2i+1$

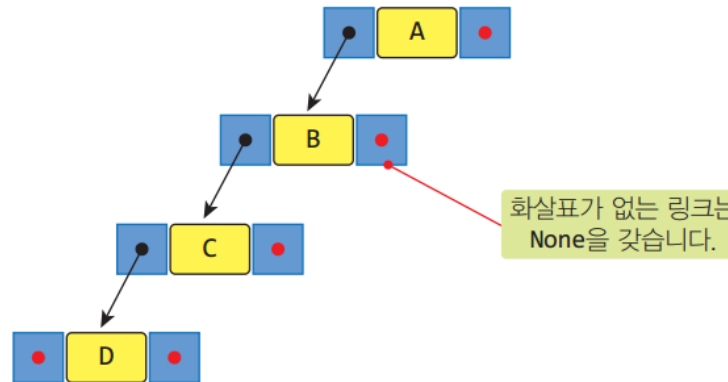
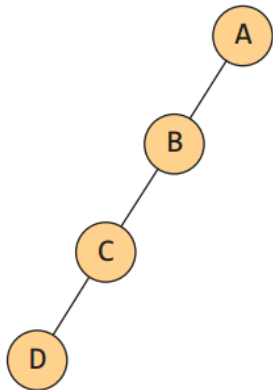
파이썬에서는 나눗셈 연산자가 /와 //로 구분되어 있습니다. 정수 나눗셈을 위해서는 $i//2$ 를 써야 합니다.

이진트리(binary tree)의 표현

- 연결리스트 표현법



완전이진트리의 링크 표현



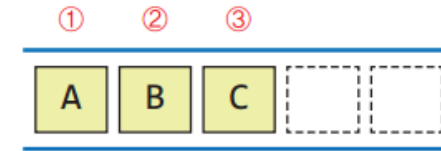
화살표가 없는 링크는
None을 갖습니다.

경사이진트리의 링크 표현

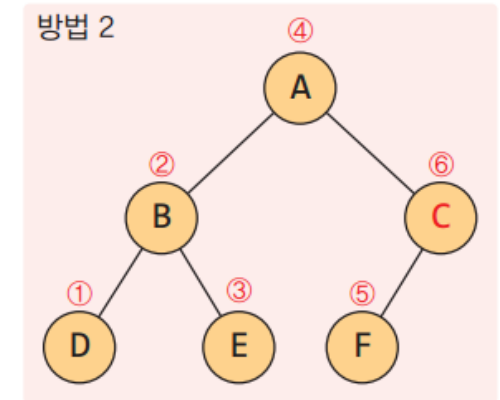
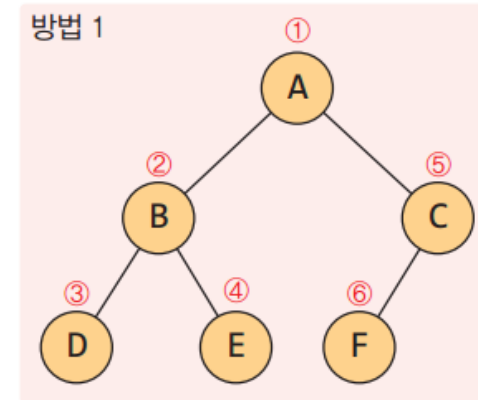
```
class TNode:
    def __init__(self, data, left, right):
        self.data = data
        self.left = left
        self.right = right
```

순회(traversal)

- 트리에 속하는 모든 노드를 한 번씩 방문하는 것
- 이진트리에서 수행되는 기본 연산들은 트리를 순회(Traversal)하며 이루어진다.
- 이진트리의 4가지 순회하는 방식
- 방식은 각각 다르지만 순회는 항상 트리의 루트노드부터 시작
 - 전위순회(Preorder Traversal)
 - 중위순회(Inorder Traversal)
 - 후위순회(Postorder Traversal)
 - 레벨순회(Levelorder Traversal)



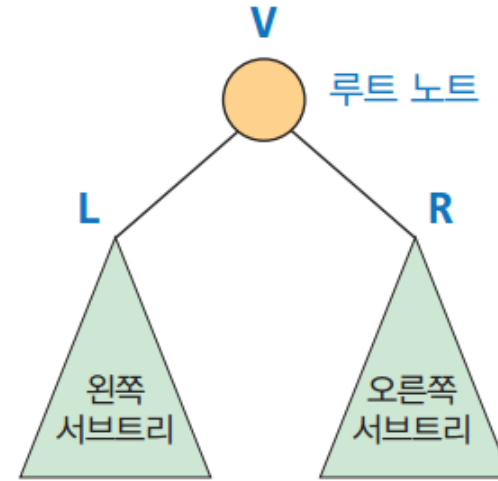
선형자료구조는
순회 방법이 단순하다.



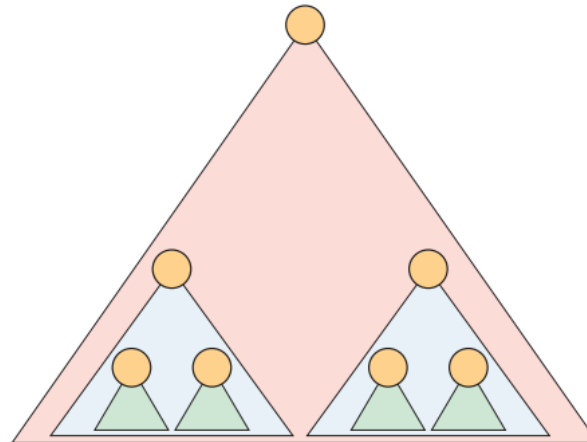
트리는 다양한 방법으로 순회할 수 있다.

이진 트리의 기본 순회(traversal)

- 이진트리의 기본 순회
 - 전위 순회(preorder traversal) : VLR
 - 중위 순회(inorder traversal) : LVR
 - 후위 순회(postorder traversal) : LRV

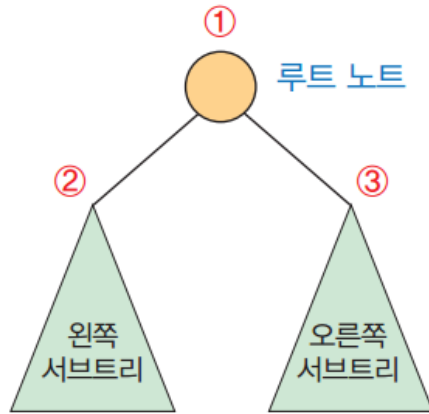


- 전체 트리나 서브 트리나 구조는 동일



전위 순회(preorder)

- 루트 → 왼쪽 서브트리 → 오른쪽 서브트리



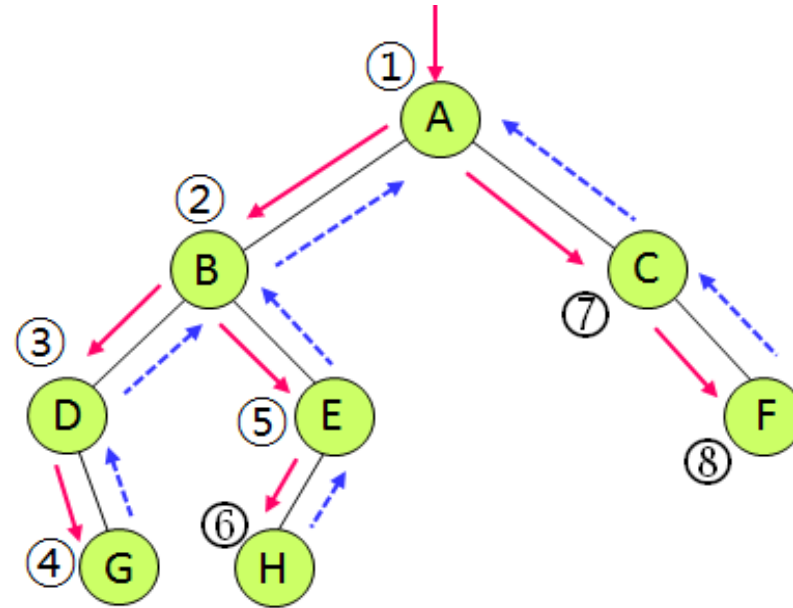
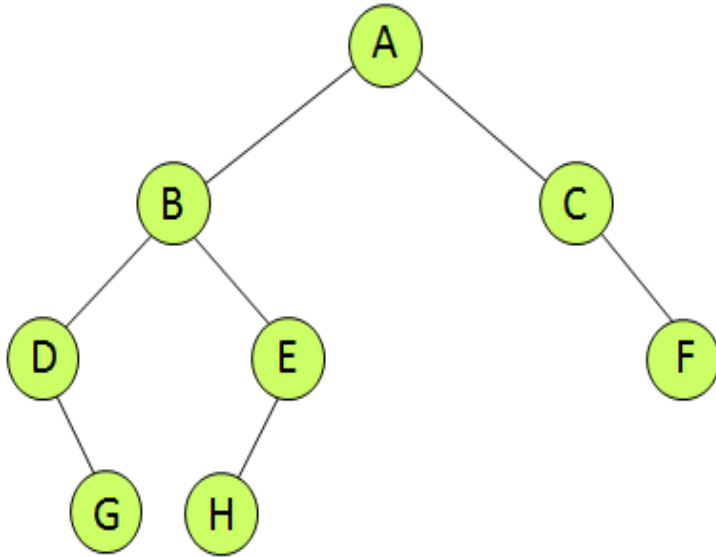
```

def preorder(n) :           # 전위 순회 함수
    if n is not None :
        print(n.data, end=' ') # 먼저 루트노드 처리(화면 출력)
        preorder(n.left)      # 왼쪽 서브트리 처리
        preorder(n.right)     # 오른쪽 서브트리 처리
  
```

- 응용 예
 - 노드의 레벨 계산
 - 구조화된 문서 출력

Binary tree

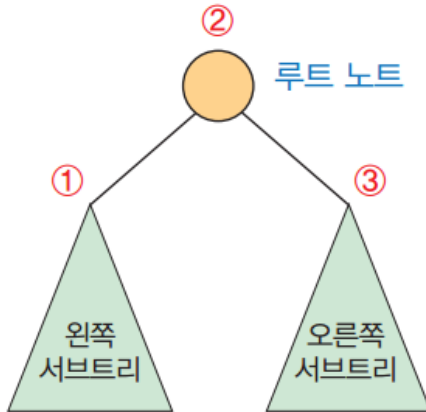
전위 순회(preorder)



실선 화살표를 따라서 A, B, D, G, E, H, C, F 순으로 방문

중위 순회(inorder)

- 왼쪽 서브트리 → 루트 → 오른쪽 서브트리



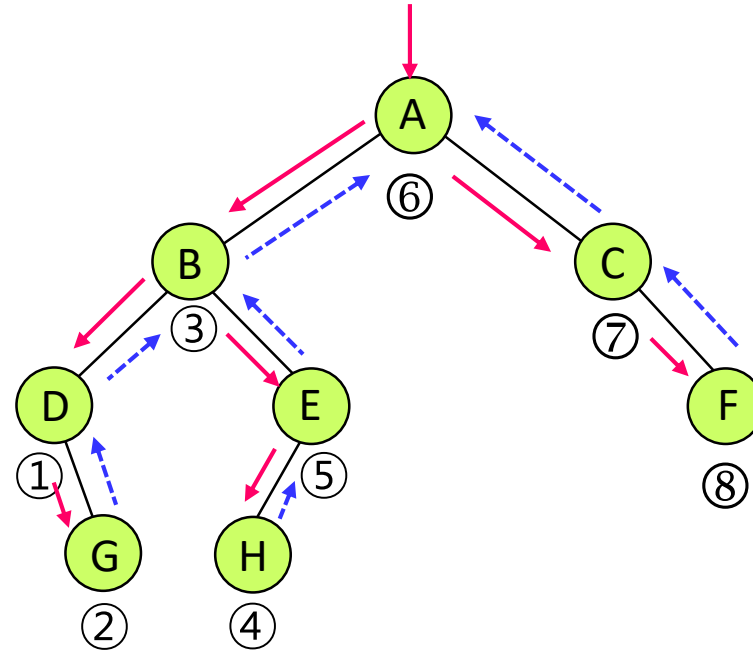
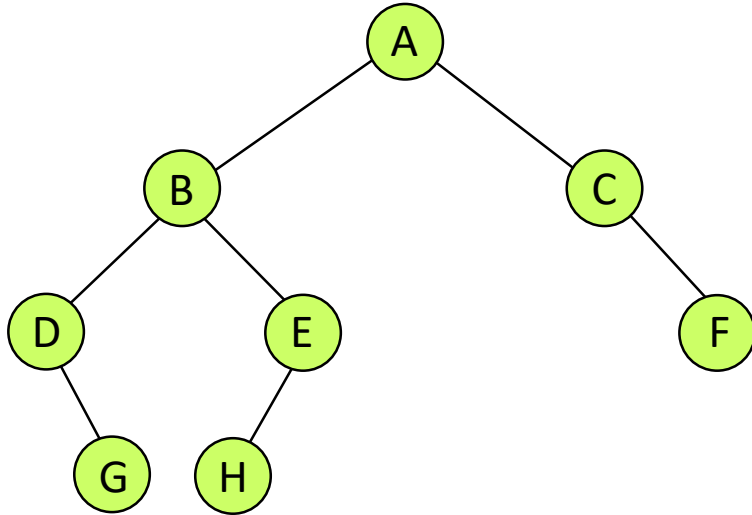
```

def inorder(n) :                                # 전위 순회 함수
    if n is not None :
        inorder(n.left)                        # 왼쪽 서브트리 처리
        print(n.data, end=' ')                # 루트노드 처리(화면 출력)
        inorder(n.right)                       # 오른쪽 서브트리 처리
    
```

- 응용 예
 - 정렬

Binary tree

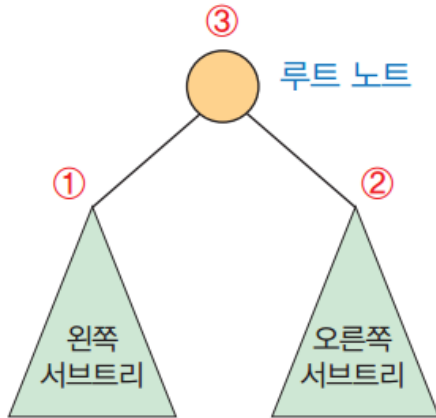
중위 순회(inorder)



중위순회: D, G, B, H, E, A, C, F 순으로 방문

후위 순회(postorder)

- 왼쪽 서브트리 → 오른쪽 서브트리 → 루트

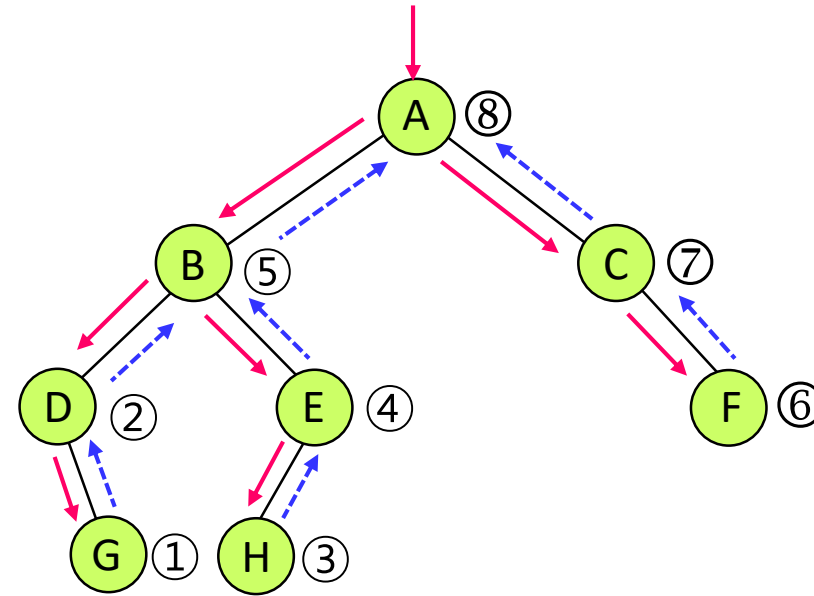
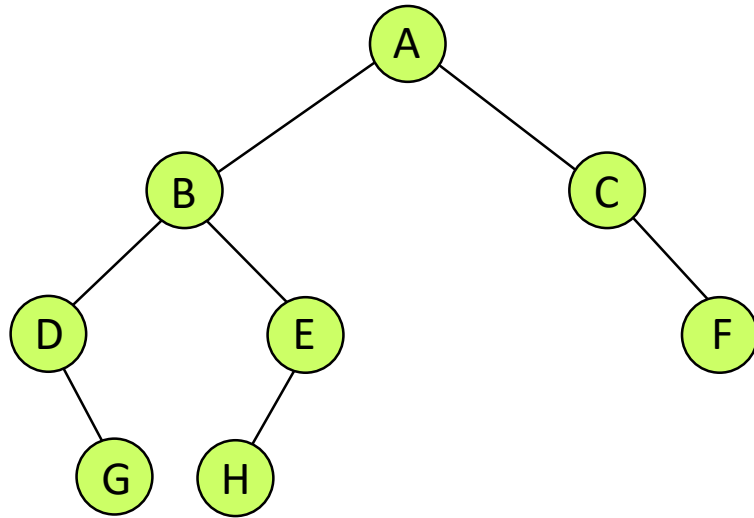


```
def postorder(n) :  
    if n is not None :  
        postorder(n.left)  
        postorder(n.right)  
        print(n.data, end=' ')
```

- 응용 예
 - 폴더 용량 계산

Binary tree

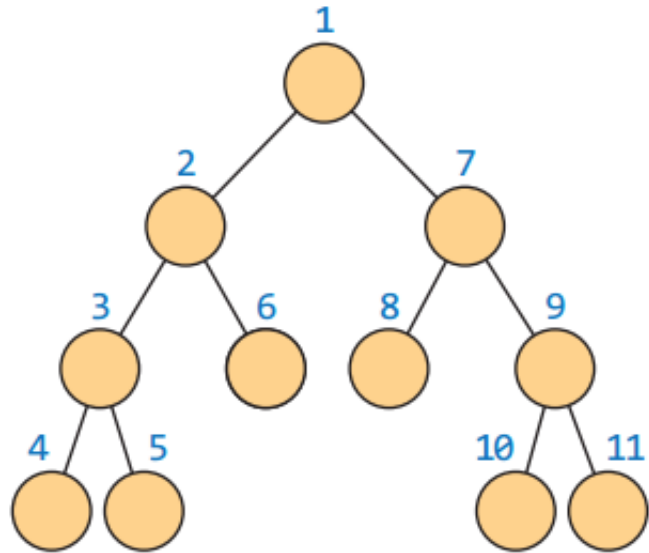
후위 순회(postorder)



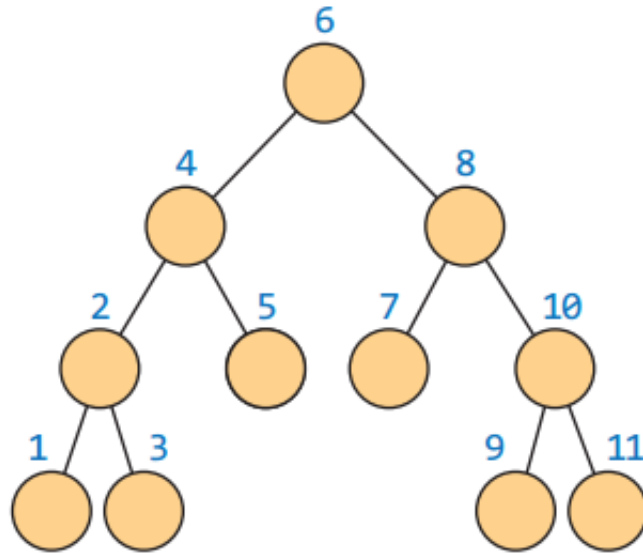
후위순회: G, D, H, E, B, F, C, A 순으로 방문

Binary tree

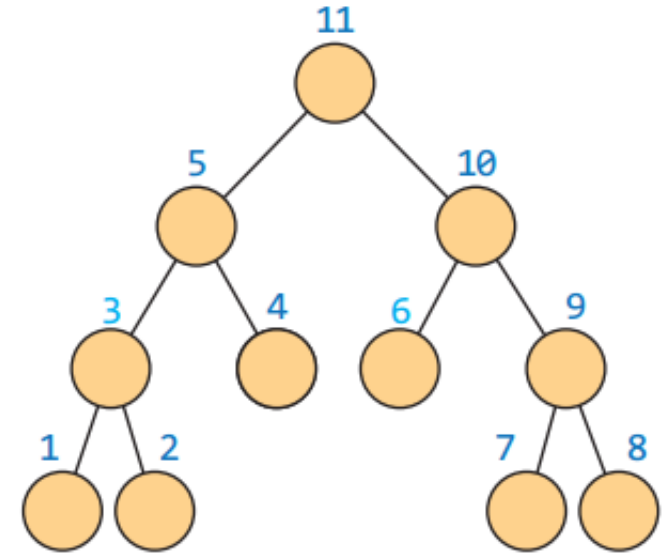
순회 방법에 따른 노드 방문 순서



(a) 전위 순회



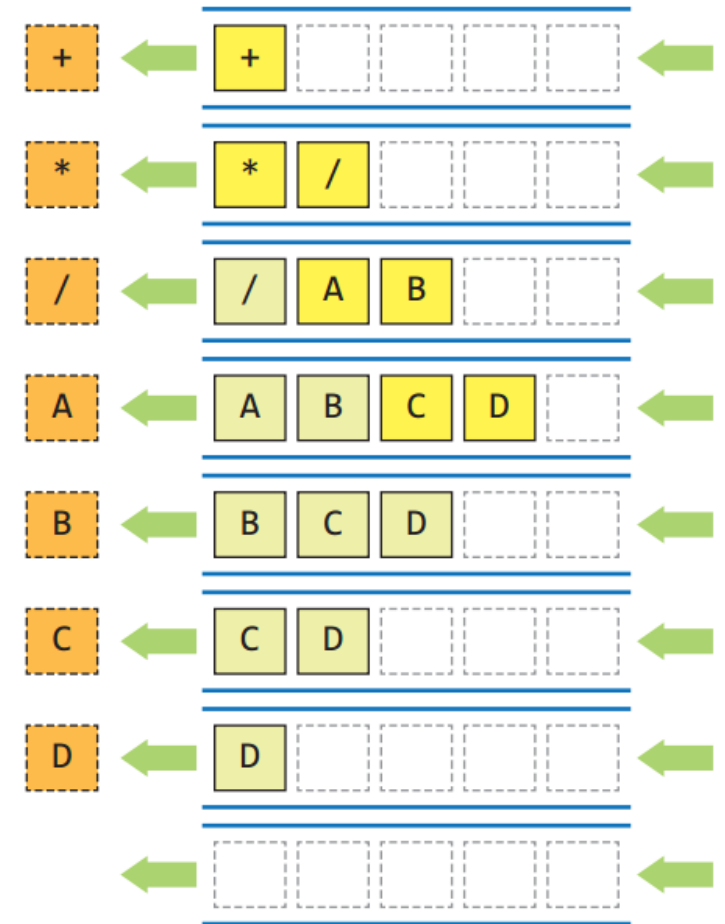
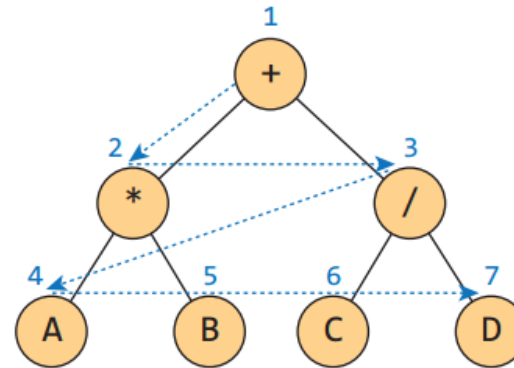
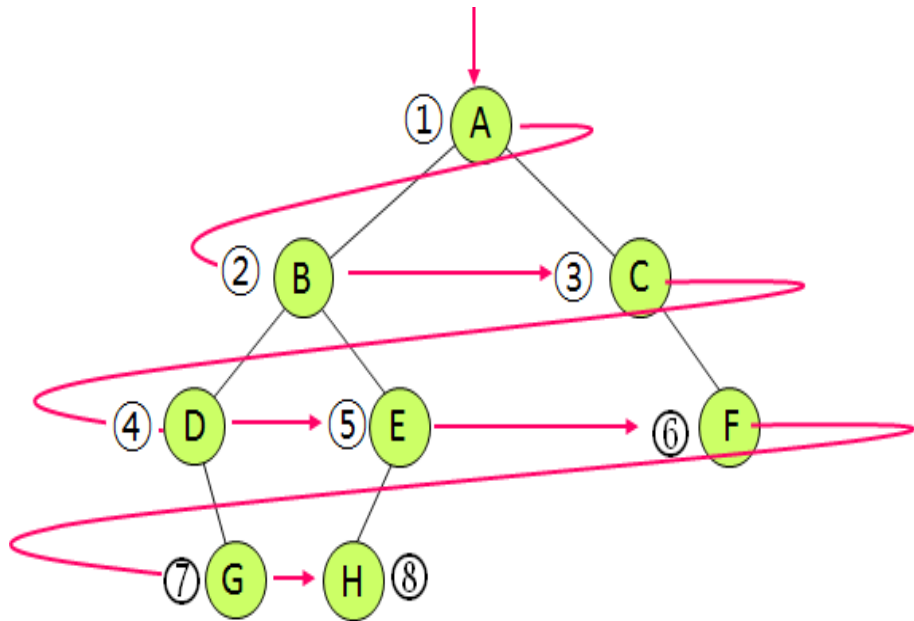
(b) 중위 순회



(c) 후위 순회

레벨 순회(levelorder)

- 루트 노드가 있는 최상위 레벨부터 시작하여 각 레벨마다 좌에서 우로 노드들을 방문
- 노드를 레벨 순으로 검사하는 순회 방법
 - 큐를 사용해 구현
 - 순환을 사용하지 않음



레벨 순회(levelorder)

```
def levelorder(root) :  
    queue = CircularQueue()  
    queue.enqueue(root)                                # 큐 객체 초기화  
    queue.enqueue(root)                                # 최초에 큐에는 루트 노드만 들어있음.  
    while not queue.isEmpty() :  
        n = queue.dequeue()                             # 큐가 공백상태가 아닌 동안,  
                                                # 큐에서 맨 앞의 노드 n을 꺼냄  
        if n is not None :  
            print(n.data, end=' ')                     # 먼저 노드의 정보를 출력  
            queue.enqueue(n.left)                       # n의 왼쪽 자식 노드를 큐에 삽입  
            queue.enqueue(n.right)                      # n의 오른쪽 자식 노드를 큐에 삽입
```

이진트리연산: 노드 개수, 단말 노드의 수

- 노드 개수

```
def count_node(n) :      # 순환을 이용해 트리의 노드 수를 계산하는 함수.  
    if n is None :      # n이 None이면 공백 트리 --> 0을 반환  
        return 0  
    else :              # 좌우 서브트리의 노드수의 합 + 1을 반환 (순환이용)  
        return 1 + count_node(n.left) + count_node(n.right)
```

- 단말 노드의 수

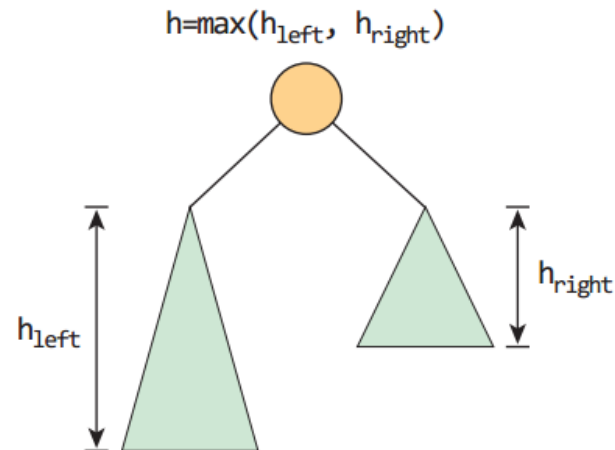
```
def count_leaf(n) :  
    if n is None :      # 공백 트리 --> 0을 반환  
        return 0  
    elif n.left is None and n.right is None : # 단말노드 --> 1을 반환  
        return 1  
    else :              # 비단말 노드: 좌우 서브트리의 결과 합을 반환  
        return count_leaf(n.left) + count_leaf(n.right)
```

이진트리연산 : 트리 높이

- 트리의 높이

```

def calc_height(n) :
    if n is None :                # 공백 트리 --> 0을 반환
        return 0
    hLeft = calc_height(n.left)   # 왼쪽 트리의 높이 --> hLeft
    hRight = calc_height(n.right) # 오른쪽 트리의 높이 --> hRight
    if (hLeft > hRight) :         # 더 높은 높이에 1을 더해 반환.
        return hLeft + 1
    else:
        return hRight + 1
    
```

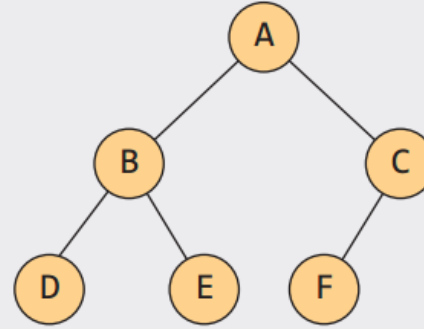


Binary tree

테스트

```

d = TNode('D', None, None)
e = TNode('E', None, None)
b = TNode('B', d, e)
f = TNode('F', None, None)
c = TNode('C', f, None)
root = TNode('A', b, c)
    
```



```

print('\n In-Order : ', end='')
inorder(root)
print('\n Pre-Order : ', end='')
preorder(root)
print('\n Post-Order : ', end='')
postorder(root)
print('\nLevel-Order : ', end='')
levelorder(root)
print()
    
```

```

print(" 노드의 개수 = %d개" % count_node(root))
print(" 단말의 개수 = %d개" % count_leaf(root))
print(" 트리의 높이 = %d" % calc_height(root))
    
```

```

In-Order : D B E A F C
Pre-Order : A B D E C F
Post-Order : D E B F C A
Level-Order : A B C D E F
노드의 개수 = 6개
단말의 개수 = 3개
트리의 높이 = 3
    
```

Binary tree

이진트리(binary tree)의 구현 및 순회

```
01 class BinaryTree:
02     class Node:
03         def __init__(self, item, left=None, right=None):
04             self.item = item
05             self.left = left
06             self.right = right
07
08     def __init__(self): # 트리 생성자
09         self.root = None
10
```

노드 생성자
항목과 왼쪽, 오른쪽 자식노드 레퍼런스

트리의 루트

이진트리(binary tree)의 구현 및 순회

```

11  def preorder(self, n): # 전위순회
12      if n != None:
13          print(str(n.item), ' ', end='')
14          if n.left:
15              self.preorder(n.left)
16          if n.right:
17              self.preorder(n.right)
18
19  def inorder(self, n): # 중위순회
20      if n != None:
21          if n.left:
22              self.inorder(n.left)
23          print(str(n.item), ' ', end='')
24          if n.right:
25              self.inorder(n.right)
26
27  def postorder(self, n): # 후위순회
28      if n != None:
29          if n.left:
30              self.postorder(n.left)
31          if n.right:
32              self.postorder(n.right)
33          print(str(n.item), ' ', end='')
34
    
```

맨 먼저 노드 방문

왼쪽 서브트리 방문 후
오른쪽 서브트리 방문

왼쪽 서브트리 방문 후
노드 방문

왼쪽과 오른쪽 서브트리
모두 방문 후 노드 방문

이진트리(binary tree)의 구현 및 순회

```

35 def levelorder(self, root): # 레벨순회
36     q = []
37     q.append(root)
38     while len(q) != 0:
39         t = q.pop(0)
40         print(str(t.item), ' ', end='')
41         if t.left != None:
42             q.append(t.left)
43         if t.right != None:
44             q.append(t.right)
45
46 def height(self, root): # 트리 높이 계산
47     if root == None:
48         return 0
49     return max(self.height(root.left), self.height(root.right))+1
    
```

리스트로 큐 자료구조 구현
 큐에서 첫 항목 삭제
 삭제된 노드 방문
 왼쪽자식, 오른쪽자식
 큐에 삽입
 두 자식노드의 높이 중 큰 높이 + 1

이진트리(binary tree)의 구현 및 순회

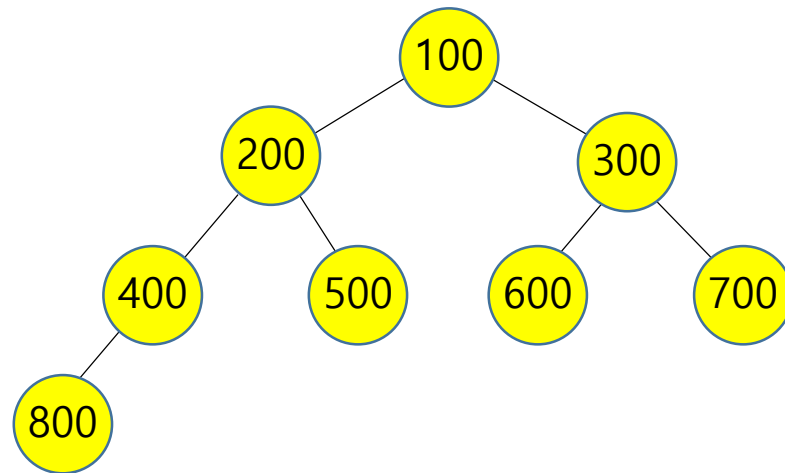
```

01 from binarytree import BinaryTree, Node
02 if __name__ == '__main__':
03     t = BinaryTree()
04     n1 = Node(100)
05     n2 = Node(200)
06     n3 = Node(300)
07     n4 = Node(400)
08     n5 = Node(500)
09     n6 = Node(600)
10     n7 = Node(700)
11     n8 = Node(800)
12     n1.left = n2
13     n1.right = n3
14     n2.left = n4
15     n2.right = n5
16     n3.left = n6
17     n3.right = n7
18     n4.left = n8
19     t.root = n1
    
```

이진트리 객체 생성

8개의 노드 생성

트리 만들기

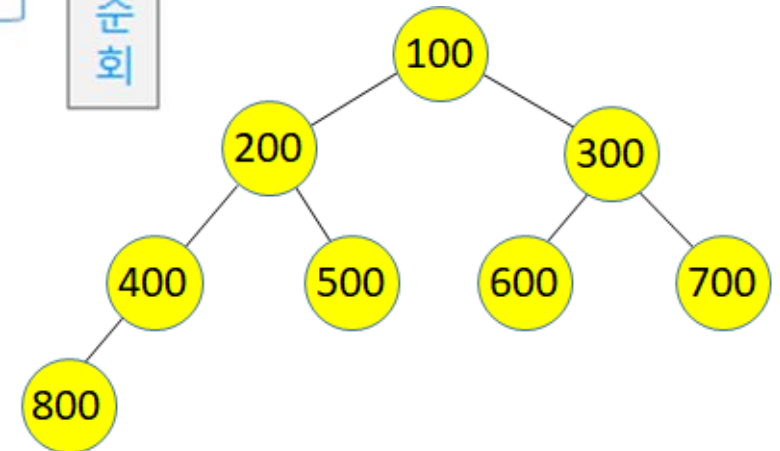


이진트리(binary tree)의 구현 및 순회

```

20     print('트리 높이 =', t.height(t.root))
21     print('전위순회:\t', end='')
22     t.preorder(t.root)
23     print('\n중위순회:\t', end='')
24     t.inorder(t.root)
25     print('\n후위순회:\t', end='')
26     t.postorder(t.root)
27     print('\n레벨순회:\t', end='')
28     t.levelorder(t.root)
    
```

트리 높이 및 4 가지 트리 순회

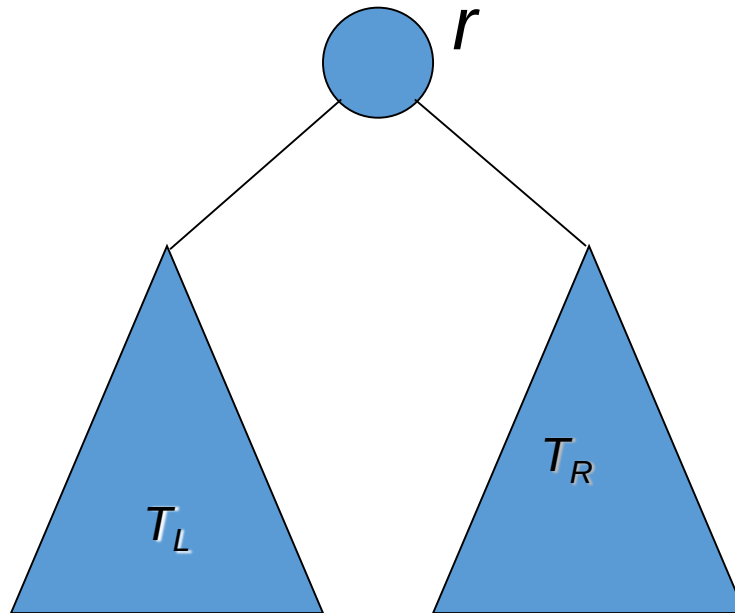


트리 높이 = 4

전위순회:	100	200	400	800	500	300	600	700
중위순회:	800	400	200	500	100	600	300	700
후위순회:	800	400	500	200	600	700	300	100
레벨순회:	100	200	300	400	500	600	700	800

이진 검색 트리(Binary search tree)

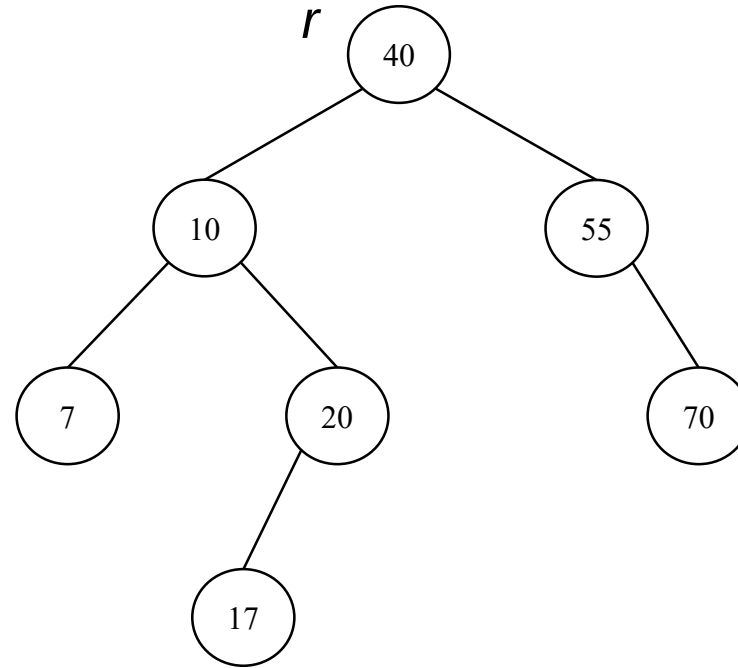
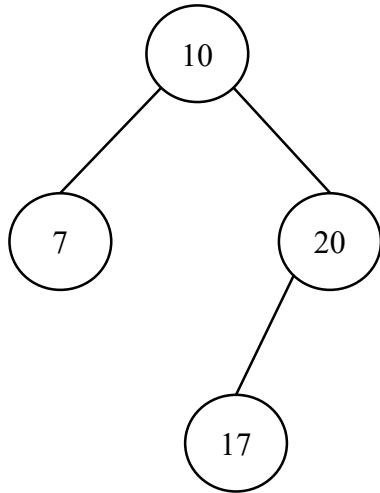
- 노드는 모두 서로 다른 키를 갖는다
- 각 노드는 최대 2개의 자식을 갖는다(최대 두 분기)
- 임의의 노드의 키key는
 - 자신의 좌서브트리left subtree에 있는 모든 노드의 키보다 크고,
 - 자신의 우서브트리right subtree에 있는 모든 노드의 키보다 작다



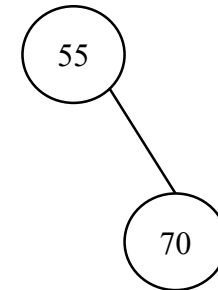
Binary tree

서브 트리(subtree)

노드 r 의 좌 서브 트리

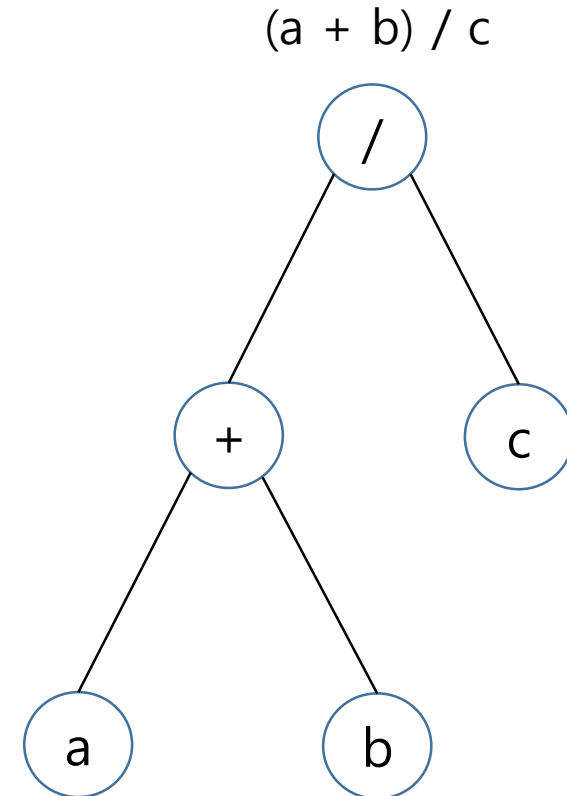
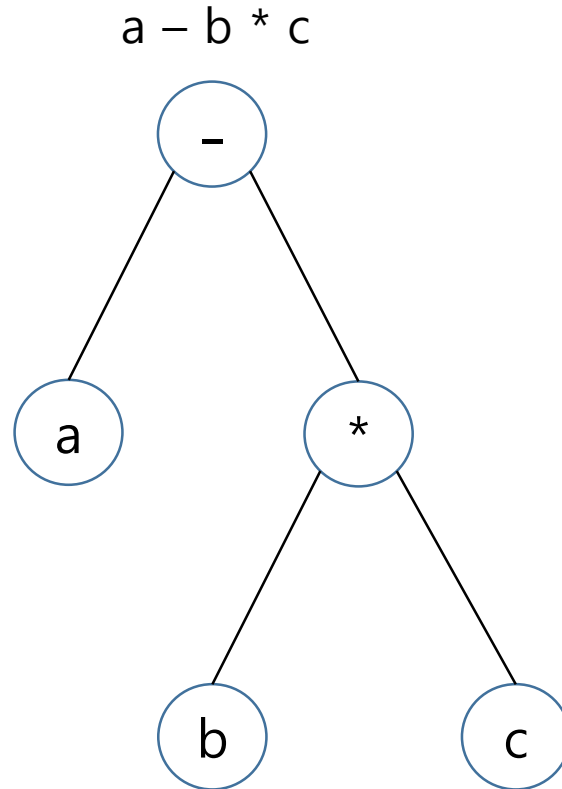
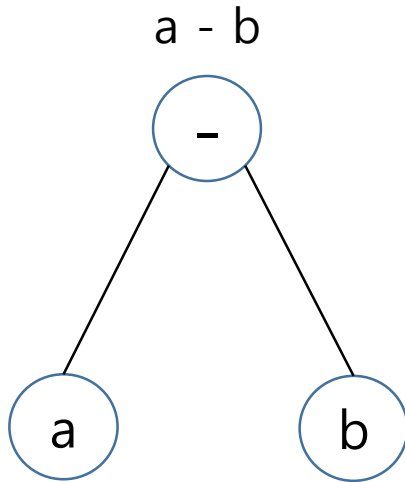


노드 r 의 우 서브 트리



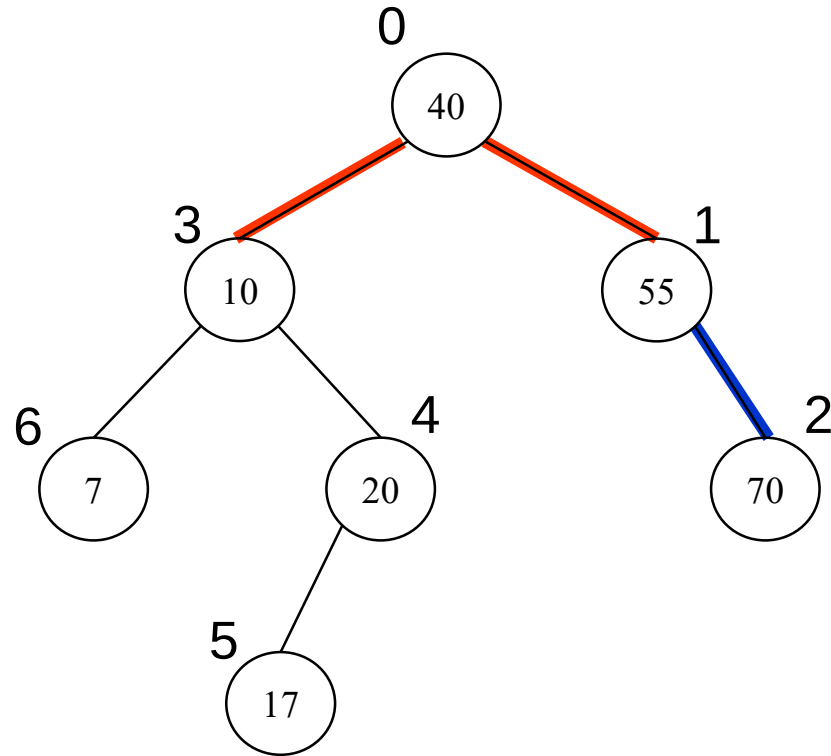
서브 트리의 예

이진트리와 수식



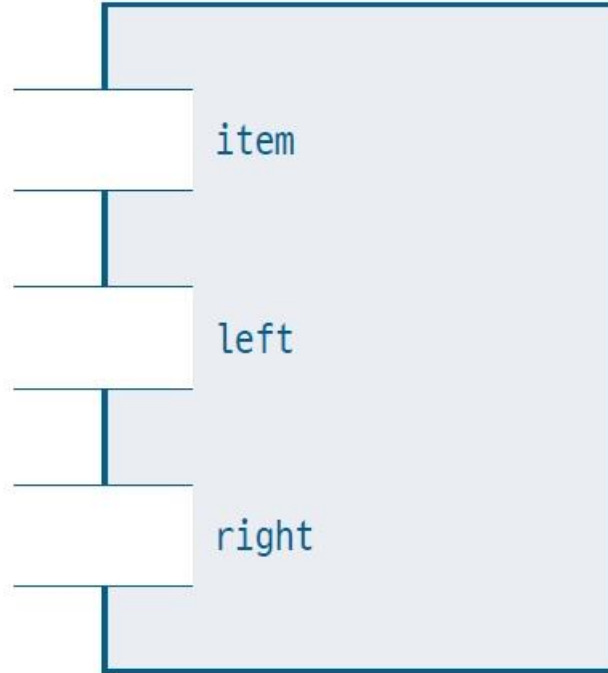
Binary tree

배열 기반의 이진 검색 트리 표현

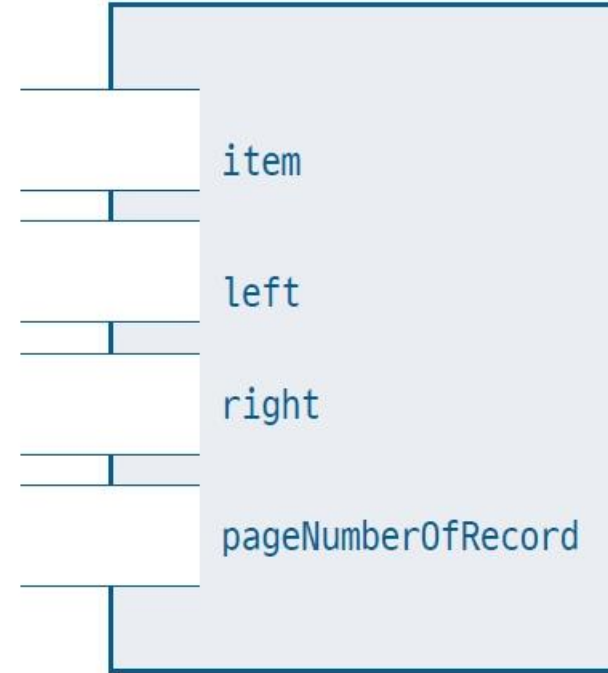


		leftChild	rightChild
0	40	3	1
1	55	-1	2
2	70	-1	-1
3	10	6	4
4	20	5	-1
5	17	-1	-1
6	7	-1	-1
7	?		
8	?		
...	...	...	...

노드 객체의 구조



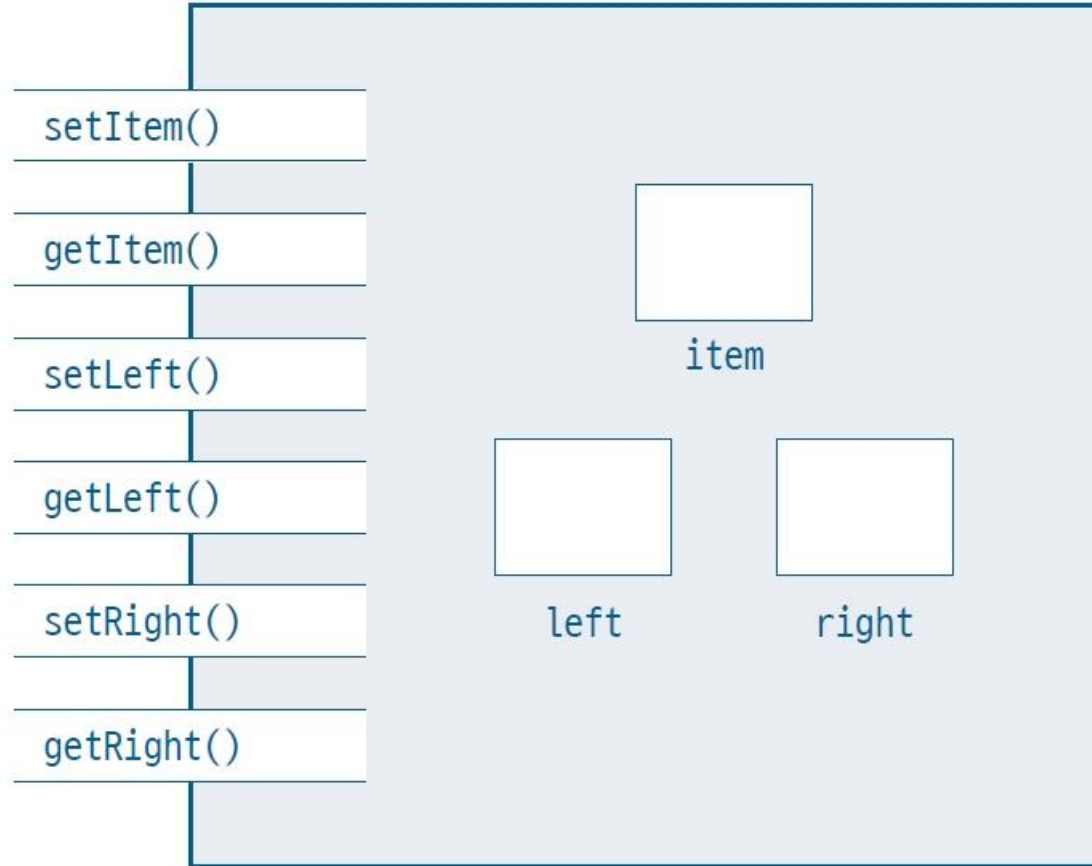
(a) 이진 검색 트리 노드



(b) 실제 이진 검색 트리 노드

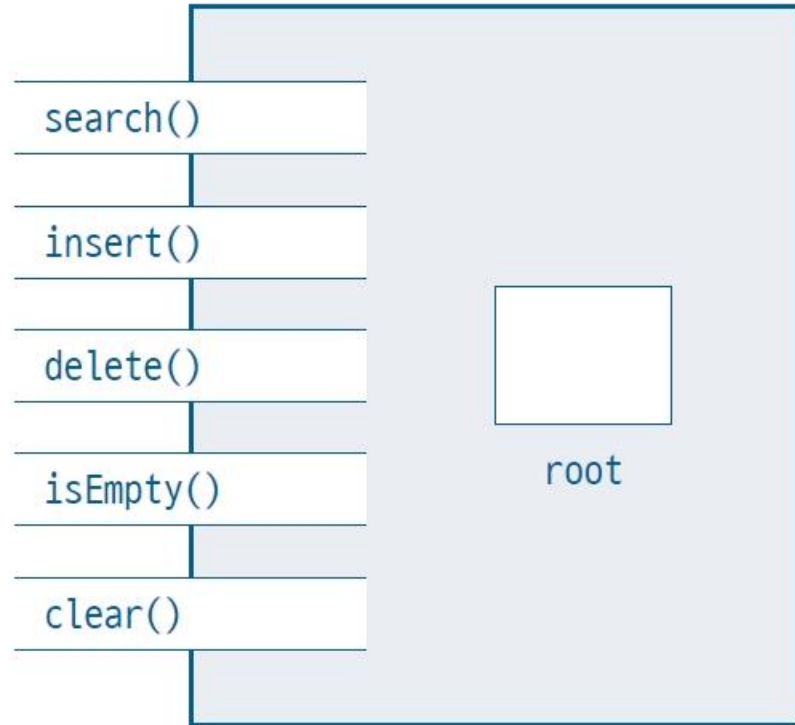
Binary tree

노드 객체의 구조



객체 지향에 최대한 충실한 이진 검색 트리의 노드 구조

노드 객체의 구조

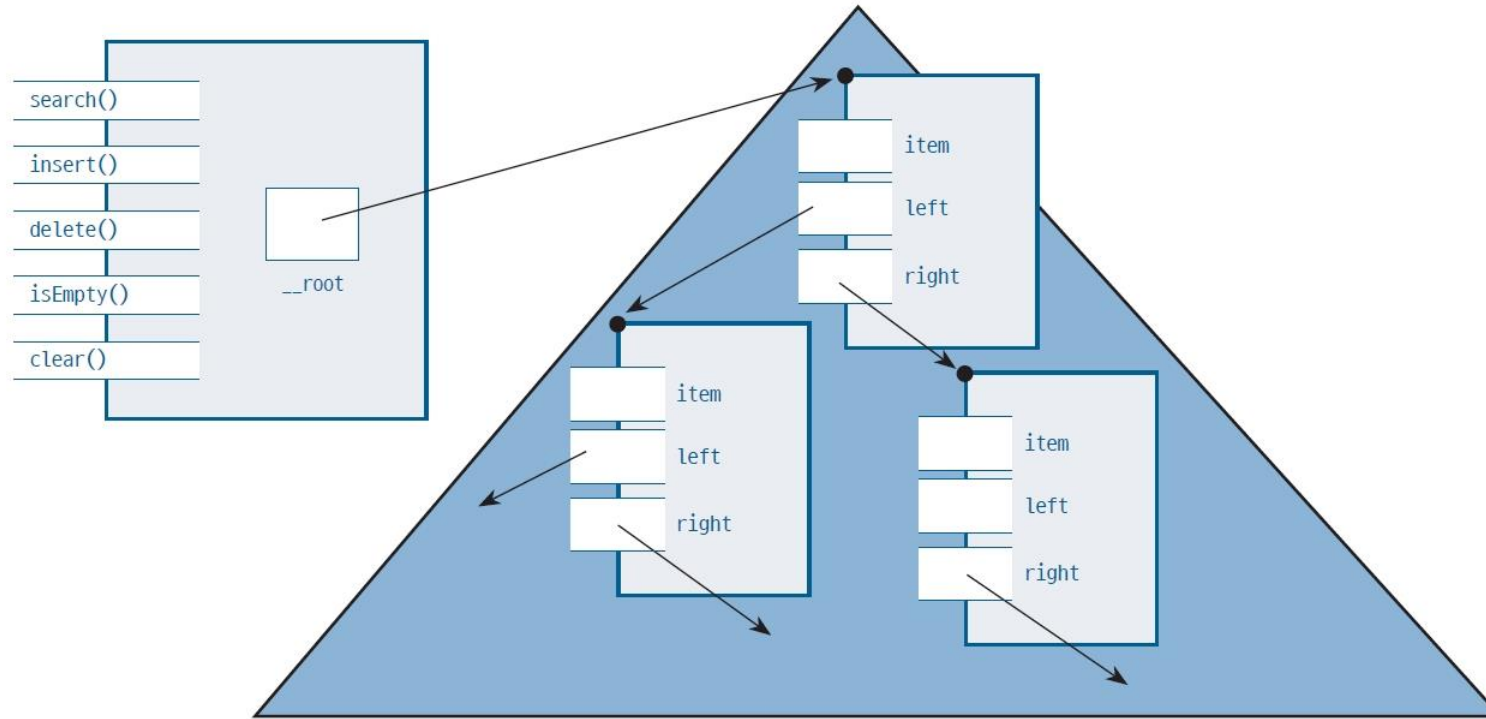


이진 검색 트리 객체 구조

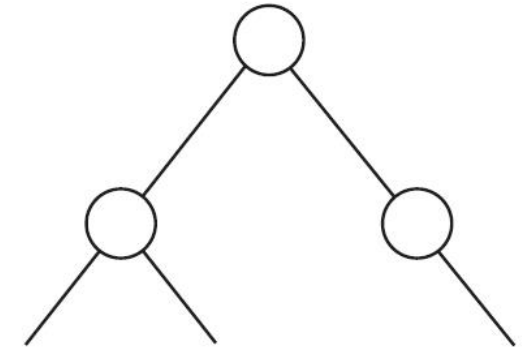
- | | |
|------------------------|-------------------------------|
| <code>__root</code> | ◀ 이진 검색 트리의 루트 노드 |
| <code>search(x)</code> | ◀ 이진 검색 트리에서 원소 x 를 검색한다. |
| <code>insert(x)</code> | ◀ 이진 검색 트리에 원소 x 를 삽입한다. |
| <code>delete(x)</code> | ◀ 이진 검색 트리에서 원소 x 를 삭제한다. |
| <code>isEmpty()</code> | ◀ 이진 검색 트리에 키가 하나도 없이 비어 있는가? |
| <code>clear()</code> | ◀ 이진 검색 트리를 깨끗이 비운다. |

Binary search tree

링크 기반의 이진 검색 트리 표현



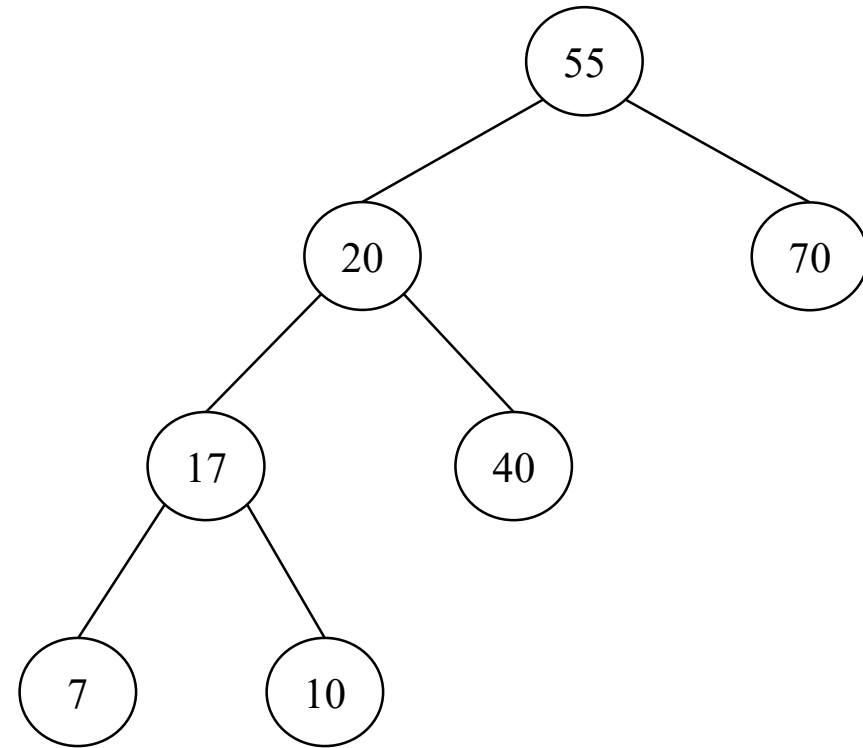
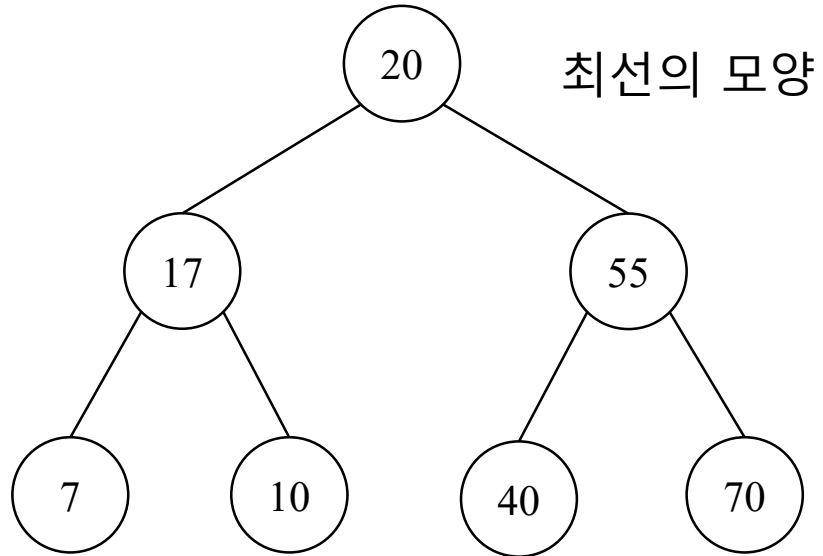
BinarySearchTree 객체와 TreeNode 객체들을 이용한 이진 검색 트리의 관계



통상적인 이진 검색 트리 시각화

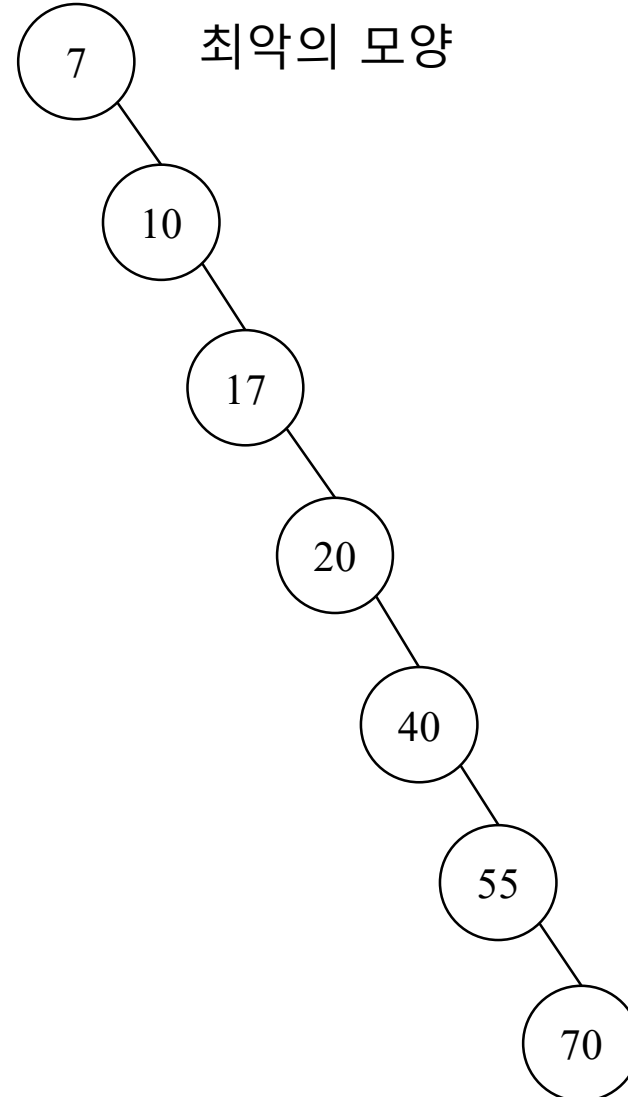
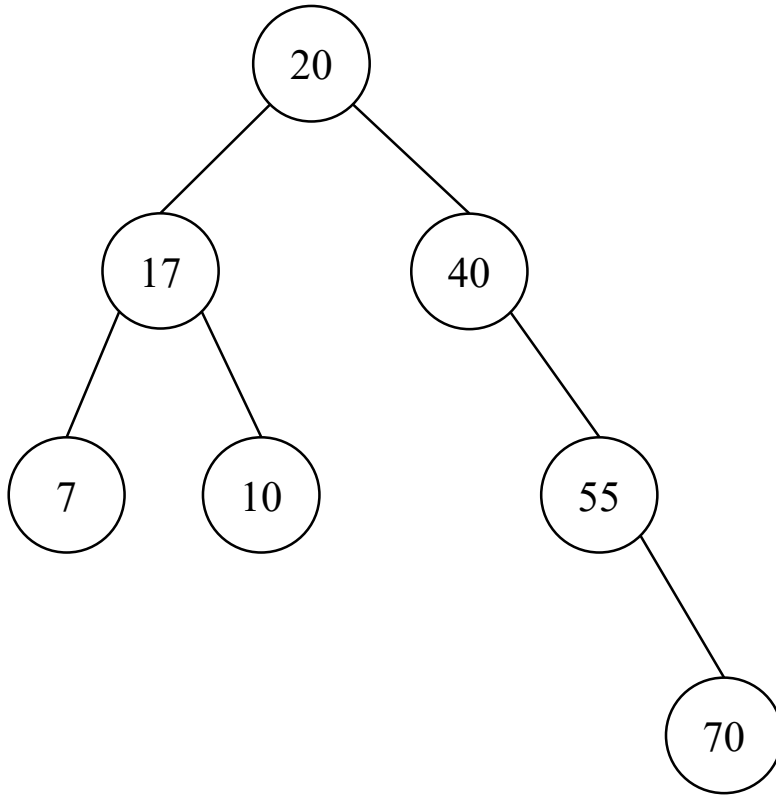
Binary search tree

같은 데이터, 다른 이진 검색 트리



Binary search tree

이런 모양이 될 수도..



Binary search tree

검색 Search

알고리즘 10-1 이진 검색 트리의 검색

search(t, x):

◀ t: (서브) 트리의 루트 노드 레퍼런스, x: 검색하고자 하는 키

① if (t = null || t.item = x)

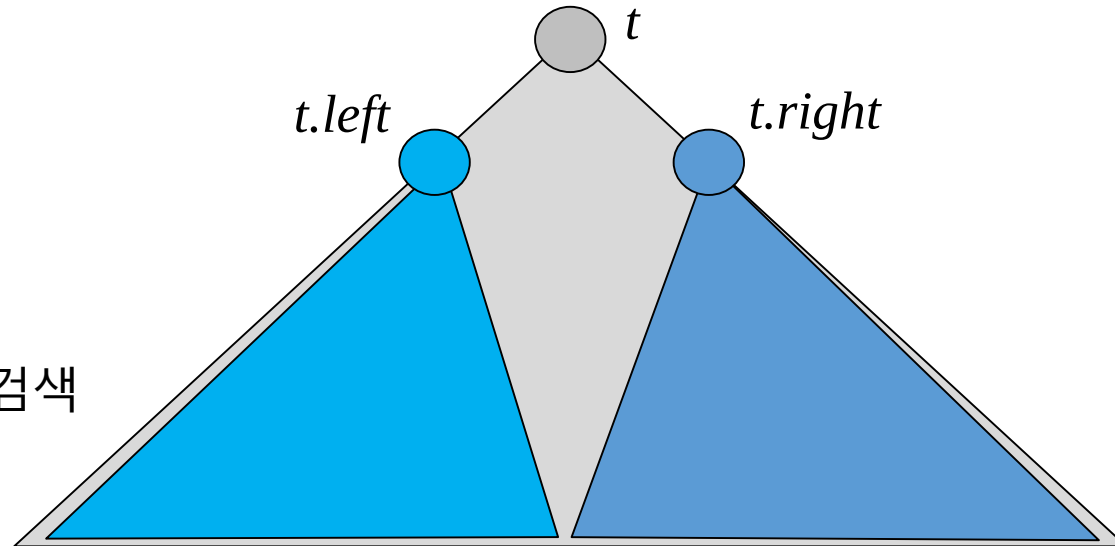
return t

② else if (x < t.item)

return search(t.left, x) ◀ t.left: t의 왼자식

③ else

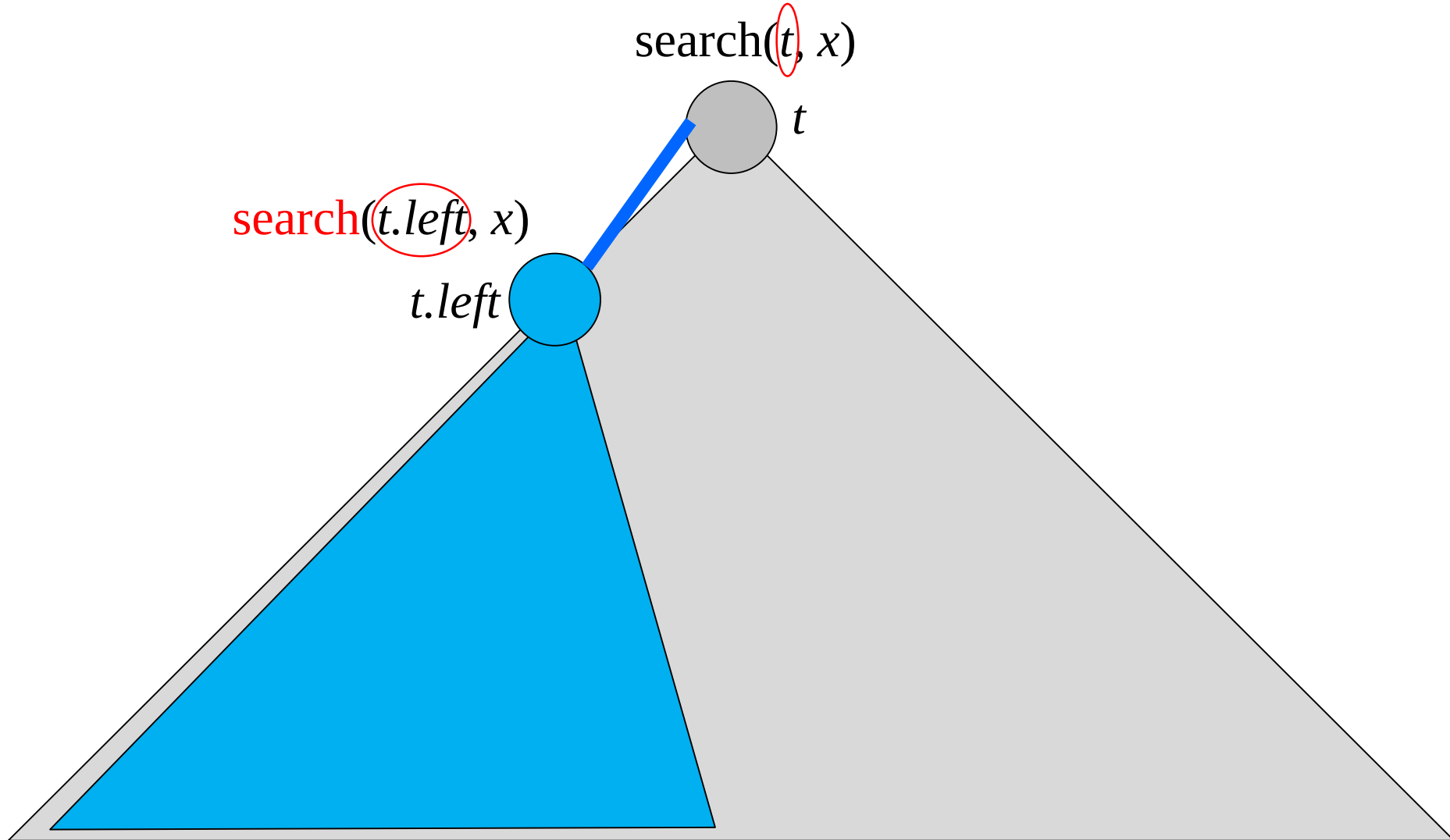
return search(t.right, x) ◀ t.right: t의 오른자식



재귀적 관점으로 본 이진 트리 검색

Binary search tree

재귀적 관점으로 본 검색

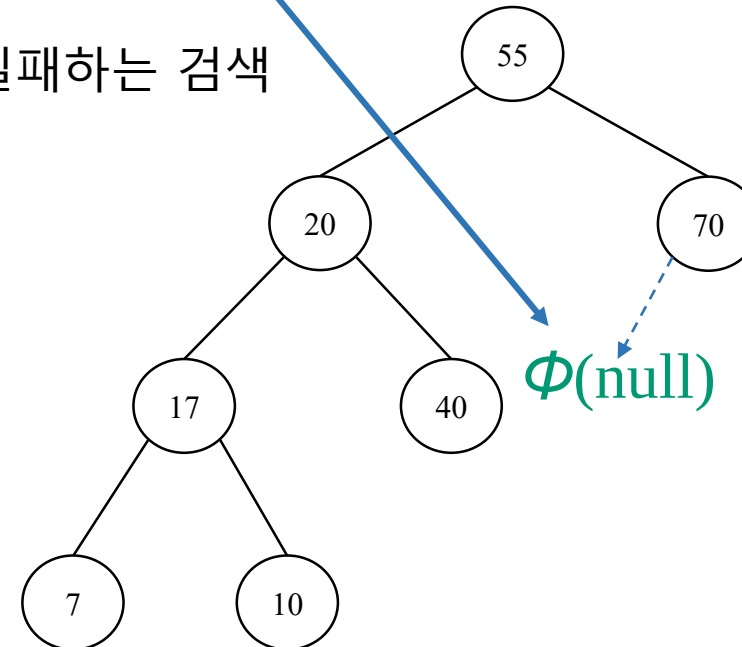


Binary search tree

검색Search

```
search(t, x):  
    if ( t = null || t.item = x )  
        return t  
    else if (x < t.item)  
        return search(t.left, x)  
    else  
        return search(t.right, x)
```

실패하는 검색

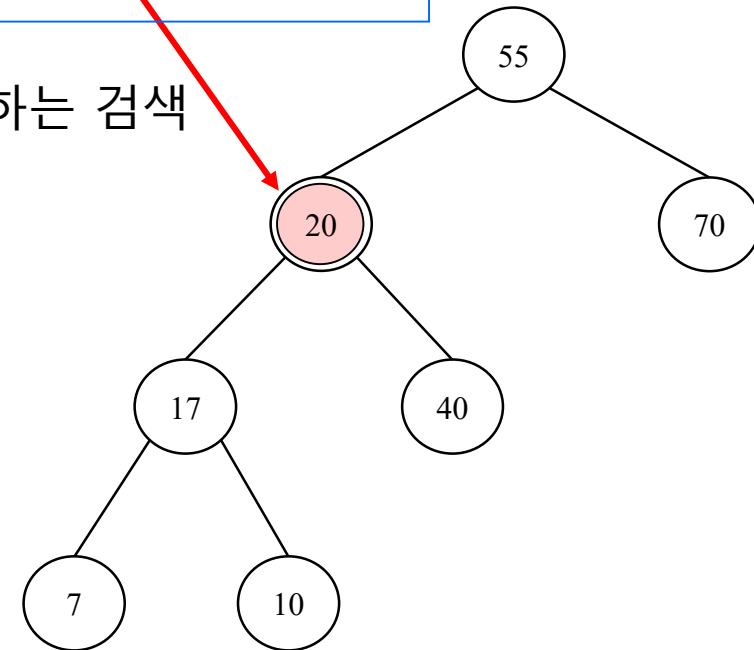


Binary search tree

검색Search

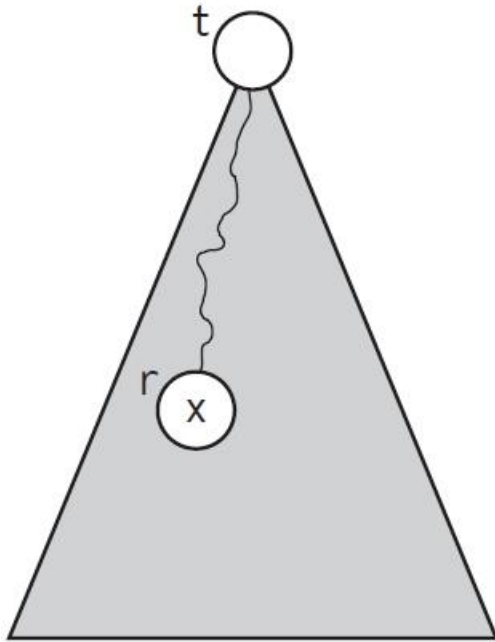
```
search(t, x):  
    if ( t = null || t.item = x )  
        return t  
    else if (x < t.item)  
        return search(t.left, x)  
    else  
        return search(t.right, x)
```

성공하는 검색

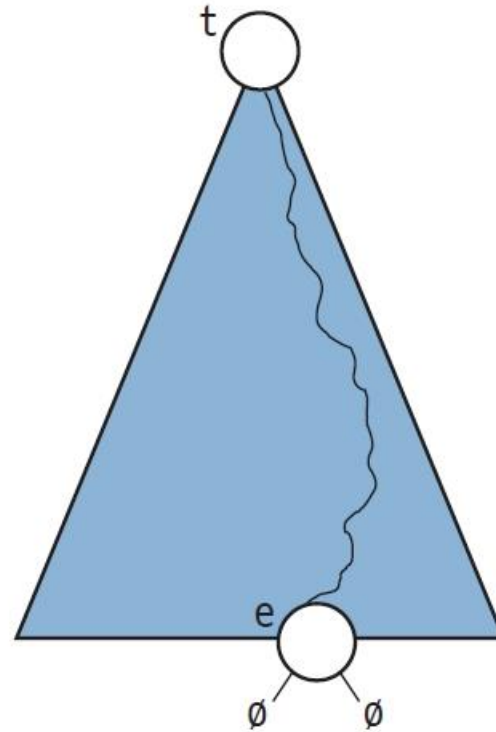


Binary search tree

검색Search



(a) 성공적인 검색



(b) 실패한 검색

· 성공적인 검색과 실패하는 검색

Binary search tree

삽입Insertion

가정: 삽입 키 x 는 검색 트리에 없다

알고리즘 10-2 이진 검색 트리의 삽입 알고리즘(스케치)

`insertSketch(t, x):`

◀ t : (서브) 트리의 루트 노드, x : 삽입하고자 하는 키

❶ if ($t = \text{null}$)

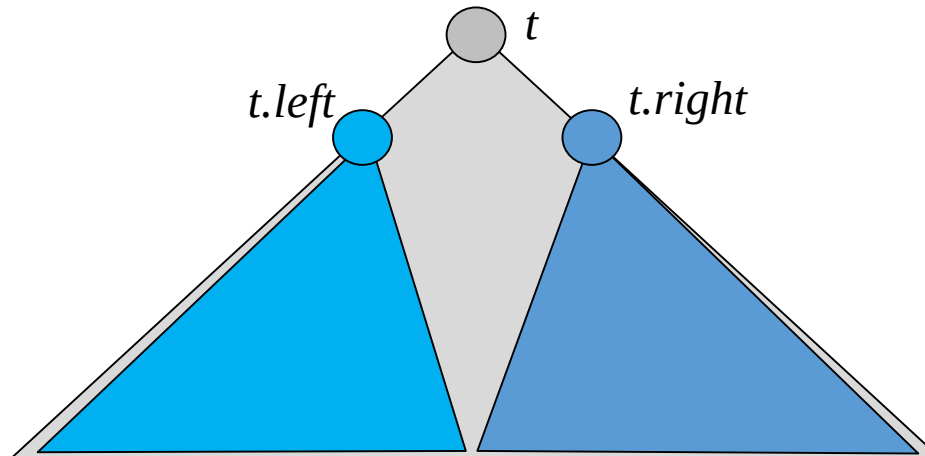
x 를 키로 하는 노드를 t 의 부모 밑에 매달고 끝낸다

❷ else if ($x < t.\text{item}$)

`insertSketch( $t.\text{left}$ ,  $x$ )`

❸ else

`insertSketch( $t.\text{right}$ ,  $x$ )`



Binary search tree

삽입Insertion

insertSketch(t, x):

- ◀ t : (서브) 트리의 루트 노드
- ◀ x : 삽입하고자 하는 키

if ($t = \text{null}$)

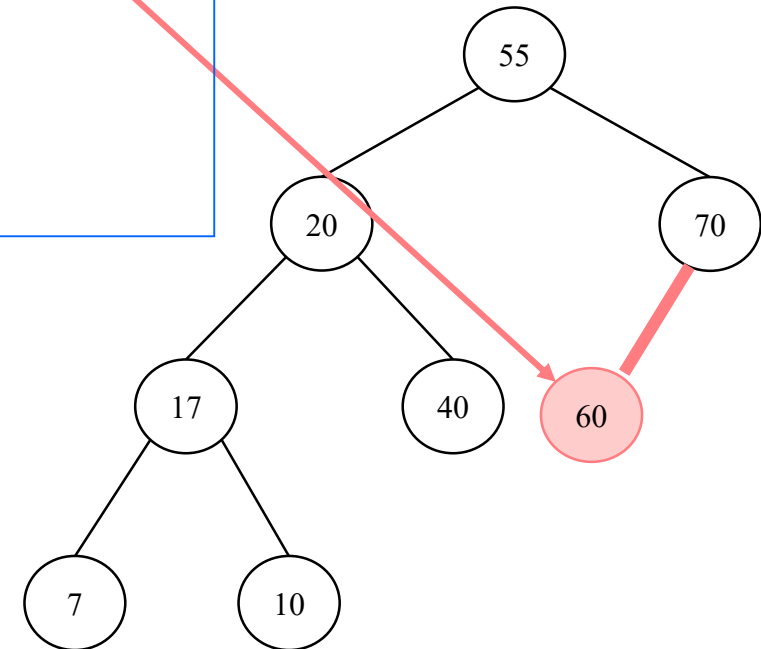
x 를 키로 하는 노드를 t 의 부모 밑에 매달고 끝낸다

else if ($x < t.\text{item}$)

insertSketch($t.\text{left}, x$)

else

insertSketch($t.\text{right}, x$)



알고리즘 insert()

알고리즘 10-3 이진 검색 트리의 삽입

insert(x):

root $\leftarrow$ insertItem(root, x) ◀ root: 트리의 루트 노드 레퍼런스

insertItem(t, x):

◀ t: (서브) 트리의 루트 노드, x: 삽입하고자 하는 키

◀ 작업 완료 후 루트 노드의 레퍼런스를 리턴한다

if (t = null)

 r.item $\leftarrow$ x; r.left $\leftarrow$ null; r.right $\leftarrow$ null ◀ r: 새 노드

 return r

else if (x < t.item)

 ❶ t.left $\leftarrow$ insertItem(t.left, x)

 return t

else

 ❷ t.right $\leftarrow$ insertItem(t.right, x)

 return t

알고리즘 insert()

알고리즘 10-4 이진 검색 트리의 삽입(비재귀적 버전)

◀ root: 트리의 루트 노드 레퍼런스

insert(x): ◀ x: 삽입하고자 하는 키

 r.item ← x; r.left ← null; r.right ← null ◀ r: 새 노드

 if (root = null)

 root ← r

 else

 t ← root

 while (t ≠ null)

 parent ← t

 if (x < t.item) t ← t.left

 else t ← t.right

 if (x < parent.item) parent.left ← r

 else parent.right ← r

삭제 Deletion

알고리즘 10-5 이진 검색 트리의 삭제 알고리즘(스케치)

```
deleteSketch(r):    ◀ r: 삭제하고자 하는 노드
    if (r이 리프 노드)                ◀ Case 1
        그냥 r을 버린다
    else if (r의 자식이 하나만 있음)  ◀ Case 2
        r의 부모가 r의 자식을 직접 가리키도록 한다
    else                            ◀ Case 3
        r의 오른쪽 서브 트리의 최소 원소 노드 s를 찾는다
        s를 r자리로 복사하고 s를 삭제한다
```

Binary search tree

삭제 Deletion

deleteSketch(r): ◀ r : 삭제하고자 하는 노드

if (r 이 리프 노드)

◀ Case 1

그냥 r 을 버린다

else if (r 의 자식이 하나만 있음)

◀ Case 2

r 의 부모가 r 의 자식을 직접 가리키도록 한다

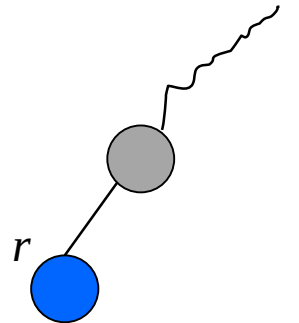
else

◀ Case 3

r 의 right subtree 의 minimum 원소 노드 s 를 찾는다

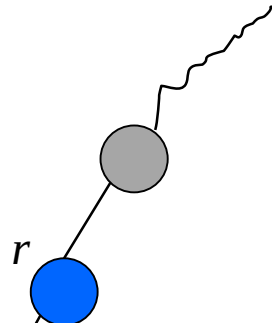
s 를 r 자리로 복사하고 s 를 삭제한다

Case 1



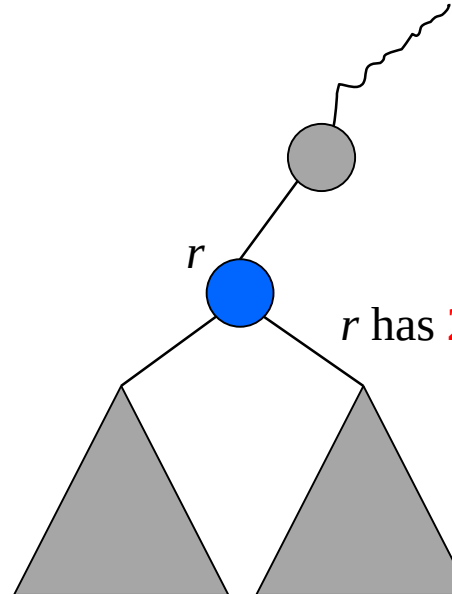
r has **no** child

Case 2



r has only **1** child

Case 3

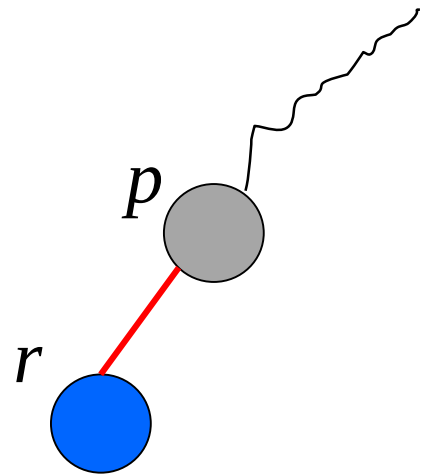


r has **2** children

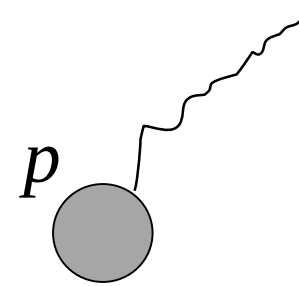
Binary search tree

삭제 Deletion

Case 1



r has **no** child

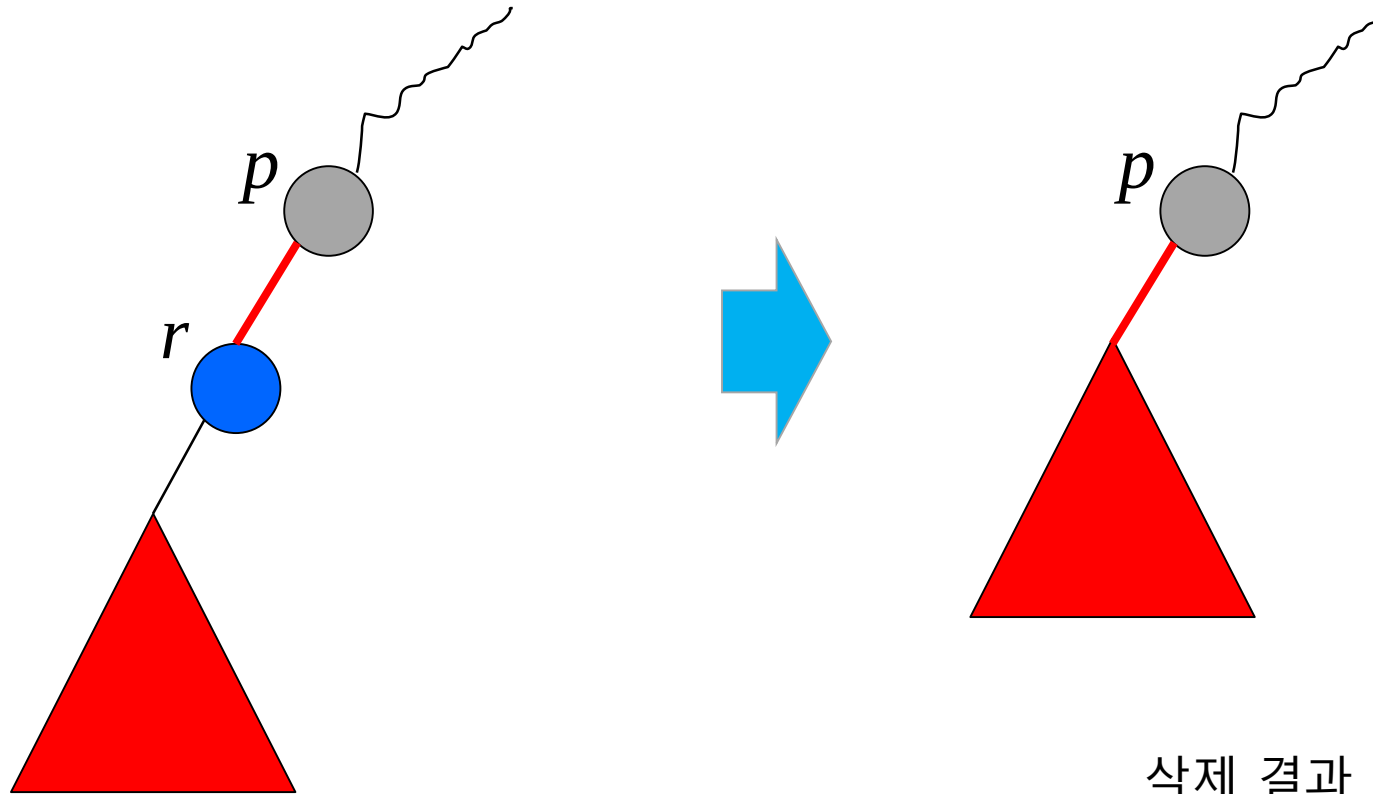


삭제 결과

Binary search tree

삭제 Deletion

Case 2



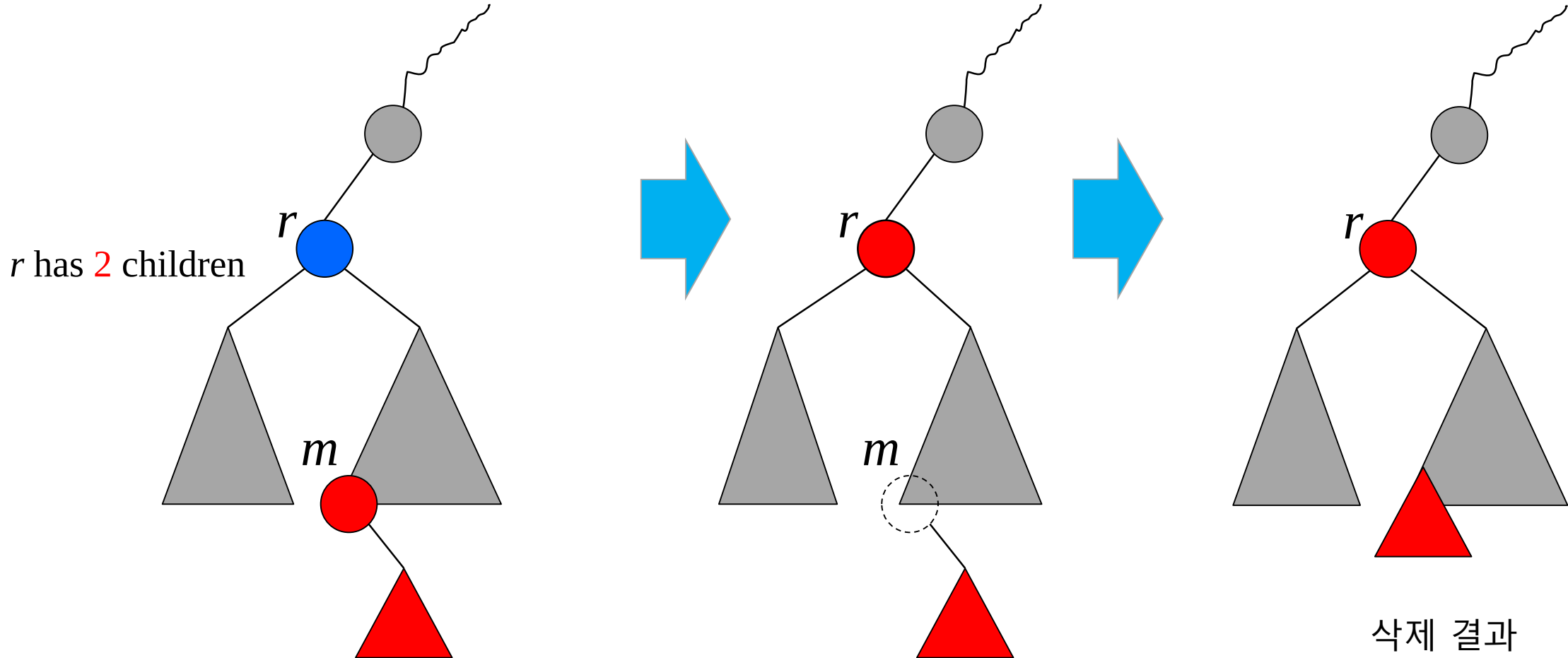
r has only 1 child

삭제 결과

Binary search tree

삭제 Deletion

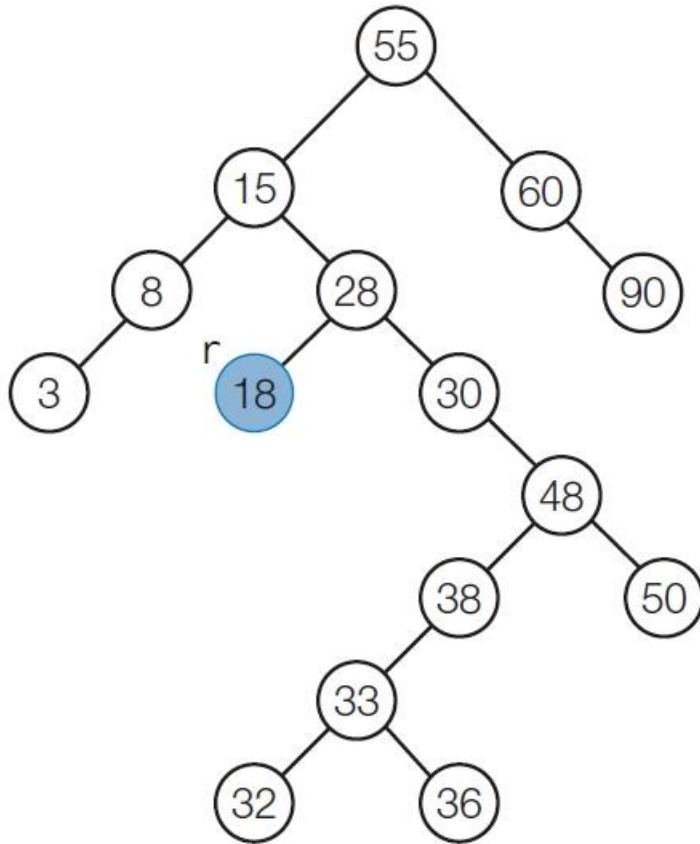
Case 3



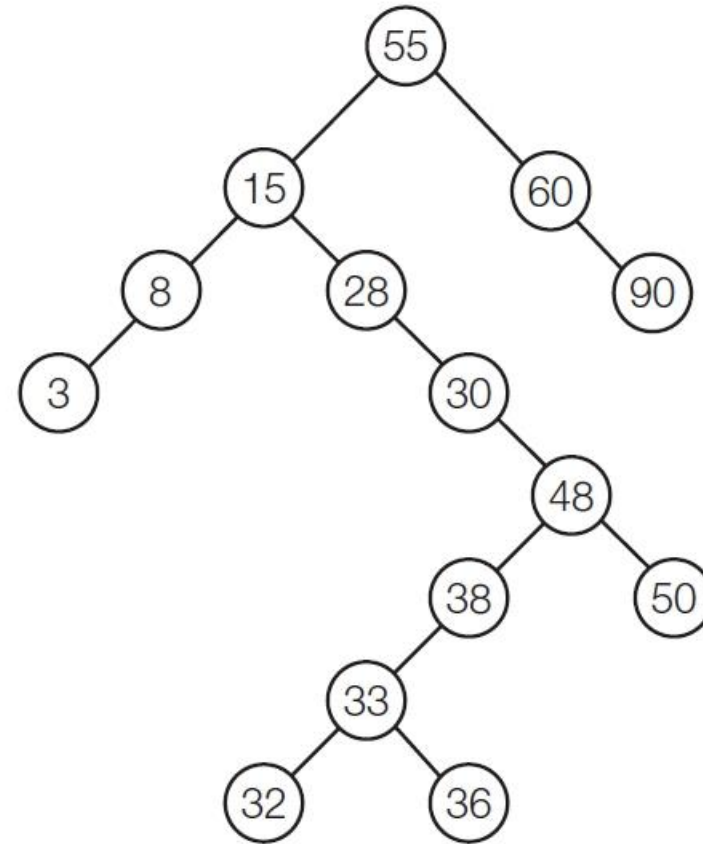
Binary search tree

삭제 Deletion

Case 1



(a) r의 자식이 없음

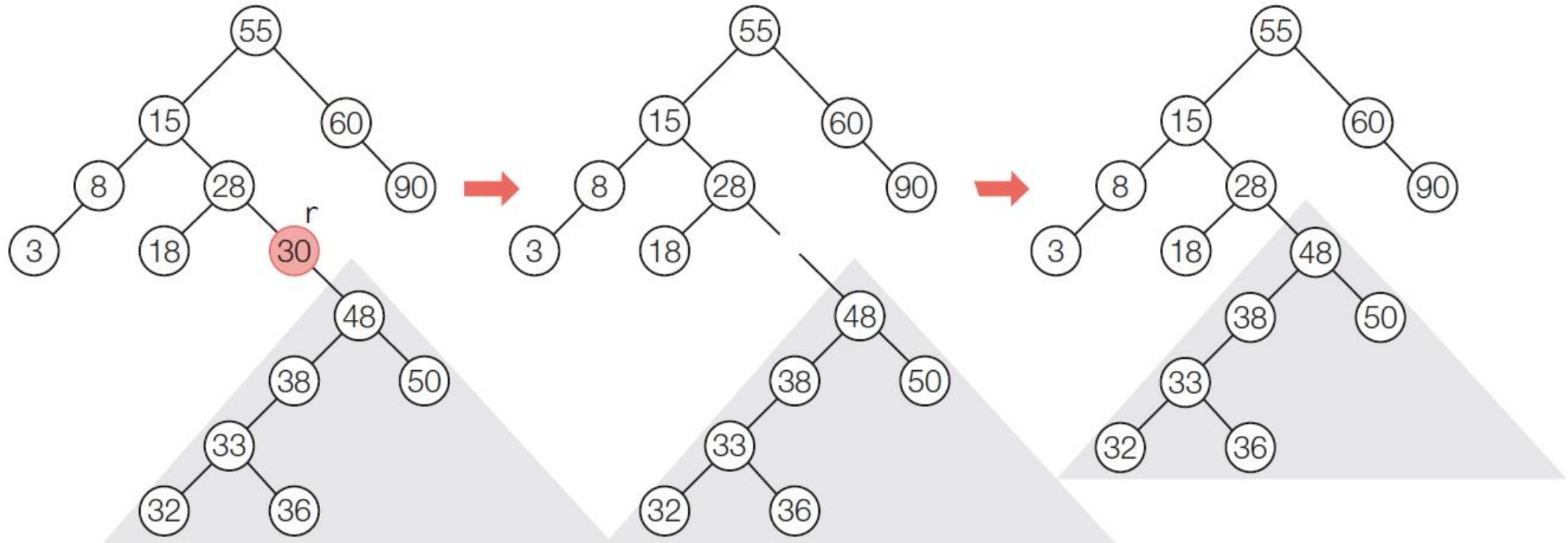


(b) 단순히 r을 제거함

Binary search tree

삭제 Deletion

Case 2



(a) r의 자식이 하나뿐임

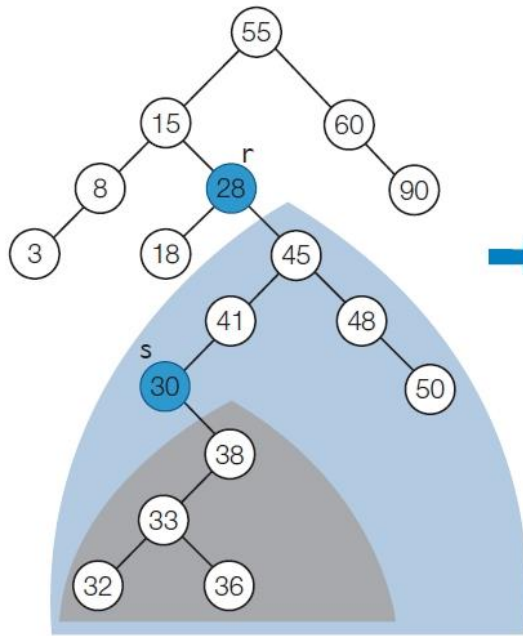
(b) r을 제거함(개념상)

(c) r 자리에 r의 자식을 놓음

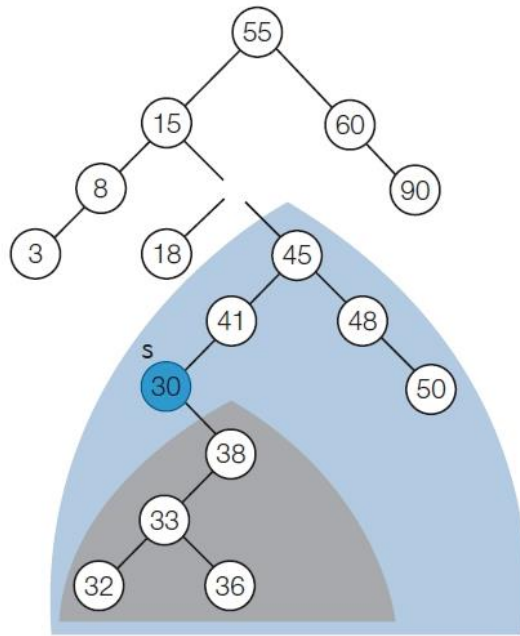
Binary search tree

삭제 Deletion

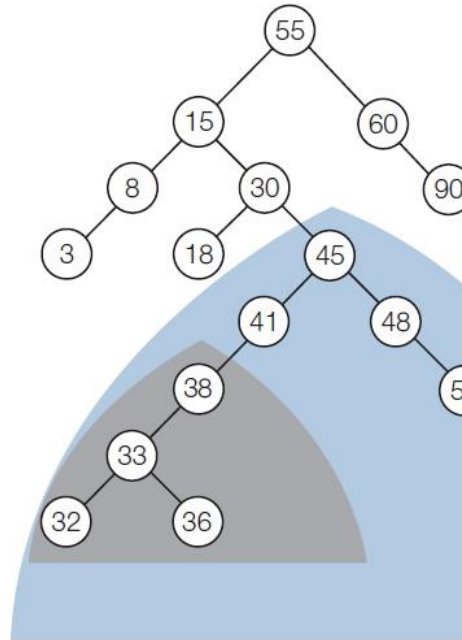
Case 3



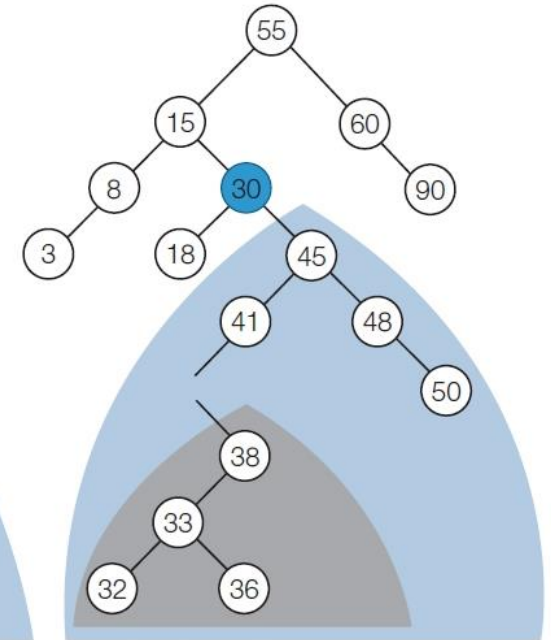
(a) r의 직후 원소 s를 찾음



(b) r을 없앴(개념상)



(d) s가 있던 자리에 s의 자식을 놓음



(c) s를 r자리로 옮김

Binary search tree

알고리즘 delete()

알고리즘 10-6 이진 검색 트리의 삭제(비재귀적 버전)

```

◀ root: 트리의 루트 노드
delete(r, p):  ◀ r: 삭제하고자 하는 노드,  p: r의 부모 노드
  ◀ r이 루트 노드인 경우
  if (r = root)
    root ← deleteNode(root);
  ◀ r이 루트 노드가 아닌 경우
  else if (r = p.left)
    p.left ← deleteNode(r)  ◀ r이 p의 왼쪽 자식
  else
    p.right ← deleteNode(r) ◀ r이 p의 오른쪽 자식

deleteNode(r):
  if (r.left = r.right = null) return null          ◀ Case 1
  else if (r.left = null and r.right ≠ null) return r.right  ◀ Case 2-1
  else if (r.left ≠ null and r.right = null) return r.left   ◀ Case 2-2
  else                                                  ◀ Case 3
    s ← r.right
    while (s.left ≠ null)
      parent ← s; s ← s.left
    r.item ← s.item      ◀ 내용 복사
    if (s = r.right) r.right ← s.right
    else parent.left ← s.right
    return r
  
```

알고리즘 delete()

알고리즘 10-7 이진 검색 트리의 삭제(재귀적 버전 + 삭제할 원소가 주어지는 버전)

delete(t, x): ◀ t: (서브) 트리의 루트 노드, x: 삭제 원소

```
if (t = null)
    에러("item not found!")
else if (x = t.item)
    t ← deleteNode(t)
    return t
else if (x < t.item)
    t.left ← delete(t.left, x)
    return t
else
    t.right ← delete(t.right, x)
    return t
```

deleteNode(t):

```
if (t.left = null && t.right = null)
    return null    ◀ case 1
```

Binary search tree

알고리즘 delete()

```
else if (t.left = null)
    return t.right                ◀ case 2 (왼자식뿐)
else if (t.right = null)
    return t.left                 ◀ case 2 (오른자식뿐)
else                             ◀ case 3 (두 자식이 다 있음)
    ① (minItem, node) ← deleteMinItem(t.right)
    t.item ← minItem
    t.right ← node
    return t                      ◀ t survived but changed

deleteMinItem(t):
    if (t.left = null)
        return (t.item, t.right)    ◀ found minimum at t
    else                             ◀ branch left
        (minItem, node) = deleteMinItem(t.left)
        t.left ← node
        return (minItem, t)
```

Binary search tree

예제 소스

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
class Binary_Search_Tree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        if self.root is None:
            self.root = Node(data)
        else:
            self.base = self.root
            while True:
                if data == self.base.data:
                    print("중복된 KEY 값")
                    break
                elif data > self.base.data:
                    if self.base.right is None:
                        self.base.right = Node(data)
                        break
                    else:
                        self.base = self.base.right
                else:
                    if self.base.left is None:
                        self.base.left = Node(data)
                        break
                    else:
                        self.base = self.base.left
```

Binary search tree

예제 소스

```
def search(self, data):
    self.base = self.root
    while self.base:
        if self.base.data == data:
            return True
        elif self.base.data > data:
            self.base = self.base.left
        else:
            self.base = self.base.right
    return False
```

```
def remove(self, data):
    self.searched = False
    self.cur_node = self.root
    self.parent = self.root
    while self.cur_node:
        if self.cur_node.data == data:
            self.searched = True
            break
        elif self.cur_node.data > data:
            self.parent = self.cur_node
            self.cur_node = self.cur_node.left
        else:
            self.parent = self.cur_node
            self.cur_node = self.cur_node.right
    if self.searched:
        # root를 지우는 경우
        if self.cur_node.data == self.parent.data:
            self.root = None
        else:
            # [CASE 1] 삭제하는 node가 leaf node인 경우
            if self.cur_node.left is None and self.cur_node.right is None:
                if self.parent.data > self.cur_node.data:
                    self.parent.left = None
                else:
                    self.parent.right = None
```

Binary search tree

예제 소스

```
# [CASE 2] 삭제하는 node의 자식이 하나인 경우
elif self.cur_node.left is not None and self.cur_node.right is None:
    if self.parent.data > data:
        self.parent.left = self.cur_node.left
    else:
        self.parent.right = self.cur_node.left
elif self.cur_node.left is None and self.cur_node.right is not None:
    if self.parent.data > data:
        self.parent.left = self.cur_node.right
    else:
        self.parent.right = self.cur_node.right
```


Binary search tree

예제 소스

```
# [CASE 3] 삭제하는 node의 자식이 둘인 경우
elif self.cur_node.left is not None and self.cur_node.right is not None:
    self.tmp_parent = self.cur_node.right
    self.tmp_cur = self.cur_node.right
    while self.tmp_cur.left:
        self.tmp_parent = self.tmp_cur
        self.tmp_cur = self.tmp_cur.left
    if self.tmp_cur.right is not None:
        self.tmp_parent.left = self.tmp_cur.right
    else:
        self.tmp_parent.left = None
    if self.parent.data > data:
        self.parent.left = self.tmp_cur
    else:
        self.parent.right = self.tmp_cur
    self.tmp_cur.left = self.cur_node.left
    self.tmp_cur.right = self.cur_node.right
else:
    print("존재하지 않는 데이터")
```


Binary search tree

예제 소스

```
# 전위 순회
def pre_order_traverse(self, node):
    if not node:
        return
    print(node.data, end=' ')
    self.pre_order_traverse(node.left)
    self.pre_order_traverse(node.right)

# 중위 순회
def in_order_traverse(self, node):
    if not node:
        return
    self.in_order_traverse(node.left)
    print(node.data, end=' ')
    self.in_order_traverse(node.right)

# 후위 순회
def post_order_traverse(self, node):
    if not node:
        return
    self.post_order_traverse(node.left)
    self.post_order_traverse(node.right)
    print(node.data, end=' ')
```

```
b = Binary_Search_Tree()
b.insert(31)
b.insert(15)
b.insert(41)
b.insert(12)
b.insert(18)
b.insert(40)
b.insert(51)
b.insert(11)
b.insert(13)
b.insert(16)
b.insert(19)
b.insert(17)
b.insert(20)

b.pre_order_traverse(b.root)
print()
b.in_order_traverse(b.root)
print()
b.post_order_traverse(b.root)
```


Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스
- <https://wikidocs.net/195736>